

Sommario

Capitolo 16 – THREAD.....	2
Introduzione	2
Thread POSIX (pthread)	3
Libreria Pthread	3
Identificazione di thread	4
Creazione di thread	4
Terminazione di thread.....	7
Stato di uscita: esempio 1	11
Stato di uscita: esempio 2.....	13
Joining di thread: esempio	14
Ancora sul passaggio dei parametri	16
Cancellazione di thread	18
Distacco di un thread	20
Attributi dei thread.....	21
Thread vs Processo	22
Rientranza.....	24
Thread e segnali.....	24
La chiamata clone di LINUX.....	25
Libreria Pthread	26
Esempio 1	27
Esempio 2	28
Capitolo 17 – Mutex e Variabili di Condizione	32
Introduzione	Error! Bookmark not defined.
Esempio Race Condition	34
Sincronizzazione dei thread.....	Error! Bookmark not defined.
Mutex	Error! Bookmark not defined.
Acquisizione e rilascio dei mutex.....	Error! Bookmark not defined.
Attributi dei mutex	Error! Bookmark not defined.
Esempio (no race)	36
Variabili di Condizione.....	37
Attendere e segnalare una condizione	Error! Bookmark not defined.
Tipica sequenza di azioni.....	39
Produttore Consumatore.....	42
Locking contro Waiting.....	45
Esempio (Uso dei Mutex).....	50
Capitolo 18 – Semafori Posix.....	53
Introduzione	53
Operazioni sui semafori	53
Semaforo contatore.....	53
Semaforo Binario e mutua esclusione	53
Esempio Produttore – Consumatore	54
Differenze tra meccanismi di sincronizzazione	54
Funzioni per i semafori Posix	55
Semafori Posix Basati su Nome	55
Creazione di un semaforo.....	55

Chiusura di un semaforo.....	56
Persistenza degli oggetti di IPC	56
Rimozione di un semaforo	57
sem_wait e sem_trywait.....	57
sem_post e sem_getvalue	58
Esempio	58
Problema Produttore – Consumatore	62
Variabili globali.....	63
Main	63
Produttore	65
Consumatore	66
Deadlock	66
Semafori Posix basati su memoria.....	67
sem_init e sem_destroy	67
Esempio: produttore – consumatore.....	67
Condivisione di semafori tra processi	71

Capitolo 16 – THREAD

Introduzione

Un **thread** rappresenta l'**unità minima di esecuzione** all'interno di un processo e include tutte le informazioni necessarie per definire il proprio **contesto di esecuzione**. Tra questi elementi si trovano l'**ID del thread**, che lo identifica univocamente nel processo, i **valori dei registri**, lo **stack**, la **politica di scheduling** con la relativa **priorità**, la **maschera dei segnali**, la variabile **errno** e altri eventuali dati specifici.

L'adozione dei thread è motivata dal fatto che, sebbene l'uso di **processi concorrenti** permetta una maggiore efficienza nell'esecuzione delle applicazioni, i **context switch** tra processi introducono un **notevole overhead**. Questo si manifesta sia a livello di **CPU**, per il salvataggio e ripristino dello stato, sia nella gestione delle **risorse allocate** e delle **interazioni tra processi**.

Tutti i thread di uno stesso processo condividono lo **spazio di indirizzamento**, il **testo del programma eseguibile**, la **memoria globale**, quella di **heap** e i **descrittori di file**, pur mantenendo un proprio **stato di esecuzione**. Quando un processo genera due **processi figli** che ereditano il suo contesto e non hanno ancora risorse proprie, il loro contesto risulta quasi identico, differendo principalmente per lo **stack** e lo **stato della CPU**. In tali casi, il cambio di contesto comporta uno scambio di informazioni spesso **ridondanti**.

I thread sfruttano questa caratteristica, poiché consentono l'esecuzione di **più flussi di controllo** utilizzando le **stesse risorse** del processo. Lo stato del processo viene quindi suddiviso in **stato delle risorse** (comune) e **stato di esecuzione** (specifico per ciascun thread). Di conseguenza, durante la commutazione tra thread è necessario scambiare solo gli **stati di esecuzione**, riducendo **significativamente l'overhead**.

L'impiego dei thread permette di sviluppare applicazioni capaci di eseguire **più compiti simultaneamente** all'interno dello stesso processo, assegnando un thread a ciascun compito. Ciò semplifica anche la gestione di **eventi asincroni**, in quanto ogni tipo di evento può essere gestito da un thread dedicato, utilizzando un **modello di programmazione sincrona**, più semplice rispetto a quello asincrono.

Infine, l'uso di thread multipli migliora il **throughput complessivo** del programma. In un processo a thread singolo, i compiti devono essere eseguiti in modo **sequenziale**, rallentando l'esecuzione. Con più thread, invece, i **compiti indipendenti** possono essere eseguiti **in parallelo**, migliorando l'**interattività** e riducendo i **tempi di risposta**, soprattutto nei programmi che gestiscono **input/output** in tempo reale.

Thread POSIX (pthread)

L'**API POSIX pthread** è uno standard per la gestione dei thread nei programmi scritti in **C**. Essa comprende circa 60 routine che permettono operazioni come la **creazione**, la **terminazione** (normale o anomala), l'attesa dei thread, la configurazione degli **attributi di scheduling** e la gestione dello **stack**. L'API fornisce anche meccanismi per la **condivisione dei dati** tramite **mutex** e per la **sincronizzazione** attraverso **variabili di condizione**.

Linux Thread

Nel sistema **Linux**, l'API pthread è stata inizialmente implementata con **LinuxThreads**, una libreria ormai deprecata a partire da **glibc 2.4**. Questa implementazione prevedeva la creazione di un **thread manager**, responsabile della gestione dei thread, e l'uso interno di segnali (come **SIGUSR1** e **SIGUSR2**) per la comunicazione. Tuttavia, i **thread non condividevano l'ID di processo** e venivano trattati dal sistema come **processi separati**, visibili singolarmente tramite strumenti come **ps**.

NPTL

La moderna implementazione è la **NPTL (Native POSIX Threads Library)**, introdotta con **glibc 2.3.2** e supportata da kernel **Linux 2.6**. Essa garantisce maggiore **conformità agli standard POSIX** e migliori **prestazioni** nel caso di un numero elevato di thread. Con NPTL, tutti i thread fanno parte dello stesso **gruppo di thread** e condividono lo stesso **PID**. Non viene creato alcun **thread manager**, e vengono usati i **primi due segnali real-time**, che quindi non sono disponibili per l'uso nelle applicazioni utente.

Entrambe le implementazioni adottano un **modello 1:1**, nel quale ogni thread utente corrisponde a un'entità gestita direttamente dal **kernel**, utilizzando la system call **clone**.

Libreria Pthread

Identificazione di thread

Ogni **thread** è identificato da un proprio **ID**, che ha significato solo all'interno del processo a cui appartiene. Questo identificatore è rappresentato dal tipo di dato `pthread_t`, la cui implementazione può variare: in alcune piattaforme è un puntatore, in altre una struttura o un intero. A causa di questa variabilità, non è possibile trattare i valori di tipo `pthread_t` come interi in modo portabile, e nemmeno stamparli direttamente in maniera universale.

```
#include <pthread.h>
int pthread_equal (pthread_t tid1, pthread_t tid2);
```

Per confrontare due ID di thread, si utilizza la funzione `pthread_equal()`, che restituisce un valore non nullo se i due ID sono uguali.

```
#include<pthread.h>
pthread_t pthread_self(void)
// restituisce l'ID del thread invocante
```

In alternativa, un thread può ottenere il proprio ID mediante la funzione `pthread_self()`. Questa funzione può essere usata con `pthread_equal()` quando un thread ha bisogno di identificare le strutture dati etichettate con il proprio ID di thread. Un thread master può impostare dei carichi di lavoro su di una coda ed usare l'ID del thread per controllare quale job va a ciascun thread di lavoro.

In ambienti diversi si possono avere implementazioni differenti: ad esempio, **Linux 2.4.22** utilizza un **long unsigned int**, mentre **Solaris 9** un intero senza segno. Inoltre, poiché `pthread_t` può essere una struttura complessa, non esiste un modo **portabile** per stamparne il valore. Tuttavia, in fase di **debugging**, può essere utile visualizzare l'ID, accettando anche un eventuale codice non portabile, senza che ciò costituisca un grave limite per lo sviluppo.

Creazione di thread

Nel modello tradizionale di **UNIX**, ogni processo dispone di un solo **thread di controllo**, rendendolo concettualmente equivalente a un processo a singolo thread. Con l'introduzione di **pthread**, l'esecuzione di un programma parte con un unico thread di controllo, e solo successivamente, se richiesto, vengono creati ulteriori thread.

```
#include<pthread.h>
int pthread_create(pthread_t *tidp, const pthread_attr_t *attr, void *(*start_rtn) (void *), void *arg)
// restituisce 0 se OK, numero di errore se fallisce
```

La creazione di un thread avviene tramite la funzione `pthread_create()`, che richiede come parametri l'indirizzo in cui salvare l'**ID del nuovo thread**, eventuali **attributi** (che possono essere impostati a `NULL` per usare quelli predefiniti), una **funzione di inizio esecuzione** (`start_rtn`) e un **argomento** (`arg`) da passare a tale funzione. Se la chiamata ha successo, l'ID del thread appena creato viene scritto nella variabile `tidp`.

Il nuovo thread inizia l'esecuzione dalla funzione `start_rtn`, che deve accettare un solo parametro (`arg`) di tipo `void*`. Se sono necessari più argomenti, è buona pratica inserirli in una **struttura** e passarne l'indirizzo.

Una volta creato, non c'è alcuna **garanzia sull'ordine di esecuzione**: il thread chiamante e quello appena generato possono partire in ordine imprevedibile. Il nuovo thread eredita lo **spazio di indirizzamento**, l'**ambiente**, e la **maschera dei segnali** del processo, ma non l'insieme dei **segnali pendenti**, che viene **azzerato**.

Le funzioni della libreria pthread, a differenza di altre **funzioni POSIX**, non modificano la variabile globale `errno` in caso di errore. Invece, restituiscono direttamente un **codice di errore**, il che rende l'**ambito dell'errore** più localizzato e legato alla funzione che l'ha causato.

Creazione di thread: esempio

L'esempio illustrato mostra la **creazione di un thread** in un programma C utilizzando la libreria **pthread**. Viene definita una funzione `thr_fn()` che rappresenta il codice da eseguire nel nuovo thread, e un'altra funzione `printids()` che stampa l'**ID del processo** e l'**ID del thread** corrente.

```
#include "apue.h"
#include <pthread.h>

pthread_t ntid;

void printids(const char *s)
{
    pid_t  pid;
    pthread_t  tid;
    pid = getpid();
    tid = pthread_self();
    printf("%s pid %lu tid %lu (0x%lx)\n", s, (pid_t) pid, (unsigned long) tid, (unsigned long) tid);
}

void * thr_fn(void *arg)
{
    printids("nuovo thread: ");
    return ((void *)0);
}
```

Nel `main()`, viene chiamata **pthread_create()** per avviare un nuovo thread. Se la chiamata ha successo, l'esecuzione prosegue nel **thread principale**, che stampa i suoi identificatori e poi entra in **sleep** per un secondo. Questo è necessario per evitare che il thread principale termini il programma prima che il nuovo thread abbia tempo di avviarsi.

```

int main(void)
{
    int err;
    err = pthread_create(&ntid, NULL, thr_fn, NULL);
    if (err!=0){
        fprintf(stderr, "non posso creare il thread: %s \n", strerror(err));
        exit(1);
    }
    printids("thread principale:");
    sleep(1);
    exit(0);
}

```

Una particolarità dell'esempio è che il **nuovo thread** ottiene il proprio ID usando `pthread_self()`, invece di affidarsi al contenuto della variabile `ntid` (scritta dal thread principale), per evitare **race condition**. Se infatti il nuovo thread fosse eseguito **prima** del completamento della `pthread_create()` da parte del chiamante, leggere `ntid` risulterebbe in un valore **non inizializzato**.

L'output del programma varia a seconda del **sistema operativo**. Su Solaris, FreeBSD e macOS, l'ID del processo resta lo stesso per entrambi i thread, mentre l'ID del thread cambia. In ambiente **LinuxThreads**, invece, i thread appaiono come **processi separati**, con PID diversi, a causa della particolare implementazione basata su `clone()`. Questo spiega anche perché, in alcuni casi, l'**ordine dell'output** non è prevedibile: dipende dalla pianificazione dei thread nel sistema operativo.

Infine, si nota che gli ID dei thread, sebbene trattati come **unsigned long**, spesso appaiono come **puntatori**, riflettendo la scelta implementativa interna del tipo `pthread_t`.

Creazione thread: passaggio parametri

```

#include<pthread.h>
#include<stdio.h>
/* parametri per char_print */
struct char_print_parms
{
    char character;
    int count;
};

void* char_print (void* parameters)
{
    /* cast del puntatore al tipo corretto */
    struct char_print_parms* p = (struct char_print_parms*) parameters;
    int i;
    for (i=0; i<p->count; ++i)
        fputc(p->character, stderr);
    return NULL;
}

```

Questo esempio mostra come gestire il **passaggio di parametri** a una funzione di thread in C usando la libreria **pthread**.

Viene definita una **struttura** `char_print_parms` che contiene i parametri necessari per la funzione `char_print`: un **carattere** da stampare e un **conteggio** che indica quante volte stamparlo. La funzione `char_print` riceve un **puntatore generico** (`void *parameters`), che viene **castato** al tipo corretto all'interno della funzione per accedere ai campi della struttura.

```
int main() {
    pthread_t tid1;
    pthread_t tid2;
    struct char_print_parms tid1_args;
    struct char_print_parms tid2_args;

    /* crea un thread per stampare 30000 'x' */
    tid1_args.character = 'x';
    tid1_args.count = 30000;
    pthread_create(&tid1, NULL, char_print, (void *)&tid1_args);

    /* crea un thread per stampare 20000 'y' */
    tid2_args.character = 'y';
    tid2_args.count = 20000;
    pthread_create(&tid2, NULL, char_print, (void *)&tid2_args);
    sleep(1);
    return 0;
}
```

Nel `main()`, si creano due strutture con parametri diversi e si passa il loro **indirizzo** come argomento alla funzione `pthread_create()`. Il cast `(void *)` è necessario per adattare il tipo della struttura al prototipo richiesto da `pthread_create`.

In questo caso, vengono creati due thread:

- Il primo stampa 30.000 volte il carattere 'x'
- Il secondo stampa 20.000 volte il carattere 'y'

Infine, viene chiamata la funzione `sleep(1)` per evitare che il processo principale termini prima che i thread completino l'esecuzione.

Questo approccio consente di **personalizzare il comportamento** di ciascun thread, passando strutture contenenti i dati necessari all'elaborazione, e costituisce una tecnica comune per **gestire parametri multipli** nei thread POSIX.

Terminazione di thread

La **terminazione di un thread** in un processo può avvenire in modi differenti, e ha conseguenze diverse rispetto alla terminazione dell'intero processo. Se un thread chiama `exit`, `_Exit` o `_exit`, oppure riceve un segnale con azione predefinita di terminazione, allora **tutto il processo termina**, coinvolgendo tutti i thread.

Un **singolo thread**, invece, può terminare senza arrestare l'intero processo in tre modi:

1. **Ritorno dalla routine di avvio:** il valore restituito diventa il codice di uscita del thread.
2. **Chiamata a `pthread_exit()`,** che consente di passare un valore di ritorno (tipicamente un puntatore generico).
3. **Cancellazione da parte di un altro thread** nello stesso processo.

```
#include <pthread.h>

void pthread_exit(void *rval_ptr);
```

Il valore restituito da `pthread_exit()` può essere recuperato da un altro thread tramite la funzione `pthread_join()`, che prende come parametri l'**ID del thread** da attendere e un **puntatore al puntatore** dove memorizzare il valore di ritorno.

```
#include<pthread.h>

int pthread_join(pthread_t thread, void **rval_ptr);

// restituisce 0 se OK, numero di errore se fallisce
```

Il thread che invoca `pthread_join()` si blocca fino a che il thread specificato non termina, sia per ritorno dalla funzione iniziale, per `pthread_exit()`, o per cancellazione. In quest'ultimo caso, il valore restituito sarà **PTHREAD_CANCELED**. Inoltre, effettuando la `join`, il thread target viene implicitamente posto nello stato **detached**, consentendo al sistema di liberarne le risorse.

Se non si è interessati al valore di ritorno del thread, è possibile passare **NULL** al posto del secondo parametro della `pthread_join()`. In tal caso, si attende comunque la terminazione del thread, ma non si ottiene alcuna informazione sul risultato finale.

È importante notare che, se si cerca di fare `pthread_join()` su un thread già **distaccato**, la chiamata fallisce restituendo **EINVAL**, poiché non è più possibile recuperare lo stato di terminazione di quel thread.

Esempio Terminazione di thread

```
#include "apue.h"
#include <pthread.h>
void *
thr_fn1(void *arg)
{
    printf("thread 1 returning\n");
    return((void *)1);
}
void *
thr_fn2(void *arg)
{
    printf("thread 2 exiting\n");
    pthread_exit((void *)2);
}
```

Entrambe queste funzioni rappresentano il **corpo dei due thread** creati nel programma. La funzione `thr_fn1` stampa il messaggio "thread 1 returning" e termina usando **return**, restituendo il valore 1 castato a `void*`. Questo valore sarà poi ricevuto dal thread principale con `pthread_join`.

La funzione `thr_fn2`, invece, stampa "thread 2 exiting" e termina con **pthread_exit**, passando come valore 2, anch'esso convertito a `void*`. A livello pratico, `return` e `pthread_exit` sono equivalenti nel comportamento finale: entrambi permettono di far terminare un thread restituendo un valore.

Tuttavia, l'uso di `pthread_exit` è più flessibile, poiché può essere chiamato **in qualsiasi punto** della funzione, anche prima della fine. È particolarmente utile quando si vuole terminare un thread in seguito a condizioni specifiche o errori, senza uscire da tutta la funzione.

```

int
main(void)
{
    int                err;
    pthread_t          tid1, tid2;
    void                *tret;
    err = pthread_create(&tid1, NULL, thr_fn1, NULL);
    if (err != 0)
        err_quit("can't create thread 1: %s\n", strerror(err));
    err = pthread_create(&tid2, NULL, thr_fn2, NULL);
    if (err != 0)
        err_quit("can't create thread 2: %s\n", strerror(err));
    err = pthread_join(tid1, &tret);
    if (err != 0)
        err_quit("can't join with thread 1: %s\n", strerror(err));
    printf("thread 1 exit code %ld\n", (long)tret);
    err = pthread_join(tid2, &tret);
    if (err != 0)
        err_quit("can't join with thread 2: %s\n", strerror(err));
    printf("thread 2 exit code %ld\n", (long)tret);
    exit(0);
}

```

Nel `main` vengono dichiarate alcune variabili: un intero `err` per gestire i codici di errore, due variabili `pthread_t` per contenere gli identificatori dei due thread (`tid1` e `tid2`), e un puntatore `void* tret` per ricevere il valore di ritorno dai thread.

Viene quindi chiamata la funzione **`pthread_create`** per creare il primo thread, che eseguirà `thr_fn1`. Se la creazione fallisce, viene stampato un errore. Lo stesso procedimento viene ripetuto per il secondo thread, che eseguirà `thr_fn2`.

Dopo la creazione, il `main` si mette in attesa della terminazione del primo thread usando **`pthread_join`**, e riceve il valore di ritorno in `tret`. Questo valore viene poi stampato, convertendolo in `long`.

Successivamente, lo stesso processo viene ripetuto per il secondo thread: `pthread_join` attende che termini e stampa il valore che esso ha restituito tramite `pthread_exit`.

Infine, il `main` termina con **`exit(0)`**, segnalando che il programma si è concluso correttamente.

OUTPUT:

```
$ ./a.out
thread 1 returning
thread 2 exiting
thread 1 exit code 1
thread 2 exit code 2
```

Stato di uscita: esempio 1

```
#include <pthread.h>
#include<stdio.h>
#include<stdlib.h>
void* thread1(void *arg) {
    int error;
    error=*(int*)arg;
    printf("Sono il primo thread. Parametro = %d\n", error);
    pthread_exit((void*)(long)error);
}
void* thread2(void *arg) {
    static long error;
    error=*(int*)(arg);
    printf("Sono il secondo thread. Parametro = %d\n", (int)error);
    pthread_exit((void*)&error);
}
```

La funzione `void* thread1` riceve un **argomento generico** `void*`, che viene **convertito in un puntatore a intero** e dereferenziato per ottenere il valore effettivo. Quel valore viene stampato, poi il thread termina con `pthread_exit`. Prima di passarlo a `pthread_exit`, il valore `int` viene **castato a long** e poi a `void*`, perché `pthread_exit` richiede un valore di tipo `void*`.

La funzione `void* thread2` fa quasi la stessa cosa, ma invece di restituire il valore direttamente, lo **salva in una variabile static long**, il cui **indirizzo** viene passato a `pthread_exit`. In questo modo, il valore di ritorno è un **puntatore** al valore `error`, e non il valore stesso.

```

int main()
{
pthread_t th1, th2;
int i1 = 1, i2=2;
void* uscita;
pthread_create(&th1, NULL, thread1, (void*)&i1);
pthread_create(&th2, NULL, thread2, (void*)&i2);
pthread_join(th1, &uscita);
printf("stato = %ld\n", (long)uscita);
pthread_join(th2, &uscita);
printf("stato = %ld\n", *(long*)uscita);
exit(0);
}

```

Nel `main`, vengono creati due interi `i1` e `i2` che vengono passati come argomenti ai due thread, tramite cast a `void*`.

Il primo `pthread_create` avvia il thread che esegue `thread1` e gli passa `&i1`.

Il secondo `pthread_create` avvia `thread2` e gli passa `&i2`.

Successivamente, il `main` si mette in attesa che i due thread terminino con `pthread_join`. Il valore restituito da ciascun thread viene salvato in `uscita`.

Per il primo thread (`thread1`), il valore restituito è **il valore stesso**, quindi basta stamparlo facendo un cast `(long)uscita`.

Per il secondo thread (`thread2`), il valore restituito è **l'indirizzo** della variabile `error`, quindi bisogna dereferenziare con `*(long*)uscita` per ottenere il valore vero.

OUTPUT previsto:

Sono il primo thread. Parametro = 1

Sono il secondo thread. Parametro = 2

stato = 1

stato = 2

Stato di uscita: esempio 2

```
void * compute_prime(void* arg) {
    static int candidate = 2;
    int n = *((int*) arg);          /* calcola l'n-esimo numero primo */
    int factor, is_prime;
    while (1) {
        is_prime = 1;
        for (factor = 2; factor < candidate; ++factor)
            if (candidate % factor == 0) {
                is_prime = 0;
                break;
            }
        if (is_prime) {
            if (--n == 0)
                return (void*)&candidate;
        }
        ++candidate;
    }
    return NULL;
}
```

Questa funzione calcola l'**n-esimo numero primo**. Riceve un argomento `arg`, che viene castato a `int*` e dereferenziato per ottenere `n`, cioè il numero di primi da contare.

La variabile `candidate` è **statica**, cioè mantiene il suo valore tra chiamate successive. Si parte da 2 e si verifica se ciascun numero è primo, controllando che non sia divisibile da nessun numero minore.

Ogni volta che si trova un numero primo, `n` viene decrementato. Quando `n` raggiunge 0, significa che è stato trovato il `n`-esimo numero primo, che viene **ritornato come puntatore**.

NB: Poiché `candidate` è `static`, è sicuro restituire il suo indirizzo, dato che la variabile **non va fuori scope** alla fine della funzione.

```
int main() {
    pthread_t tid;
    int which_prime = 5000;
    void* prime;

    pthread_create(&tid, NULL, compute_prime, (void*)&which_prime);

    /* attende il calcolo del numero primo */
    pthread_join(tid, &prime);
    printf("The %dth prime number is %d.\n", which_prime, *(int*)prime);
    exit(0);
}
```

Nel `main`, viene dichiarato un identificatore di thread `tid`, una variabile `which_prime` impostata a 5000 (quindi vogliamo il **5000° numero primo**), e un puntatore `void* prime` dove salveremo il valore restituito dal thread.

Il thread viene creato con `pthread_create`, specificando `compute_prime` come funzione da eseguire, e passando `&which_prime` come argomento.

Con `pthread_join`, il main attende che il thread termini e recupera il valore di ritorno (cioè l'indirizzo della variabile `candidate`) in `prime`.

Infine, si stampa il risultato usando `*(int*)prime` per dereferenziare e ottenere il valore effettivo del numero primo trovato.

OUTPUT atteso:

The 5000th prime number is 48611.

Joining di thread: esempio

Questo esempio mostra come creare **più thread** in C con `pthread`, passando loro parametri tramite una **struttura**, e poi **attendere la loro terminazione** usando `pthread_join`.

```
#include<pthread.h>
#include<stdio.h>
/* parametri per print_function */
struct char_print_parms
{
    char character;
    int count;
};

void* char_print (void* parameters)
{
    /* cast del puntatore al tipo corretto */
    struct char_print_parms* p = (struct char_print_parms*) parameters;
    int i;
    for (i=0; i<p->count; ++i)
        fputc(p->character, stderr);
    return NULL;
}
```

Qui viene definita una **struttura** chiamata `char_print_parms`, che serve per passare **due parametri** a un thread:

- `character`: il carattere da stampare,
- `count`: quante volte stamparlo.

La funzione `char_print` è la **funzione che il thread esegue**. Riceve un argomento generico `void*`, che viene castato al tipo corretto (`struct char_print_parms*`).

Il thread stampa il carattere indicato (`p->character`) sullo **stderr**, per il numero di volte specificato (`p->count`).

```
int main() {
    pthread_t tid1;
    pthread_t tid2;
    struct char_print_parms tid1_args;
    struct char_print_parms tid2_args;
    tid1_args.character = 'x';
    tid1_args.count = 30000;
    pthread_create(&tid1, NULL, char_print, &tid1_args);

    tid2_args.character = 'y';
    tid2_args.count = 20000;
    pthread_create(&tid2, NULL, char_print, &tid2_args);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    return 0;
}
```

Nel `main`, vengono create due strutture `tid1_args` e `tid2_args` per contenere i parametri da passare ai thread.

- Il primo thread (`tid1`) stamperà la lettera 'x' **30.000 volte**.
- Il secondo thread (`tid2`) stamperà la lettera 'y' **20.000 volte**.

Si usano **pthread_create** per avviare i due thread, assegnando loro le rispettive funzioni (`char_print`) e parametri.

Infine, con **pthread_join**, il `main` attende che entrambi i thread terminino prima di uscire.

Questo codice quindi stampa **tanti caratteri 'x' e 'y'** nello standard error (`stderr`) da due thread distinti. L'ordine in cui compaiono i caratteri può **variare ogni volta**, perché i thread lavorano in parallelo e l'esecuzione non è deterministica. Tuttavia, grazie a `pthread_join`, il programma **aspetta entrambi i thread** prima di terminare.

Ancora sul passaggio dei parametri

Il passaggio di parametri ai thread in ambiente POSIX può avvenire tramite l'uso di un **puntatore generico** (`void*`), sia in fase di creazione del thread con `**pthread_create**`, sia durante la sua terminazione tramite `**pthread_exit**`. Questo meccanismo offre una notevole flessibilità, in quanto consente di trasmettere al thread sia valori semplici che **informazioni più complesse**, per esempio mediante l'uso di strutture.

Utilizzando un puntatore, è infatti possibile passare **più dati contemporaneamente**, raggruppandoli in una **struttura** definita dal programmatore. Questa tecnica è utile quando un thread ha bisogno di accedere a più variabili o configurazioni specifiche per eseguire il proprio compito. Tuttavia, è necessario prestare particolare attenzione alla **validità della memoria** a cui il puntatore fa riferimento, specialmente quando la struttura è **allocata nello stack locale** del thread chiamante.

In tali casi, se la struttura viene passata a `**pthread_exit**` come valore di ritorno, il rischio è che lo **stack venga liberato** al termine della funzione chiamante. Quando il thread principale invoca `**pthread_join**` per attendere la conclusione del thread e accedere a quel valore di ritorno, si troverà potenzialmente ad accedere a **memoria non più valida**, con conseguente comportamento indefinito o errato.

Questo problema è particolarmente insidioso perché il programma può anche sembrare funzionare correttamente in alcune esecuzioni, ma fallire in altre, a seconda dello stato dello stack e della gestione della memoria al momento dell'accesso. Per questa ragione, quando si ha la necessità di restituire strutture da un thread, è preferibile allocarle **dinamicamente** con `**malloc**`, assicurandosi poi di **liberarle correttamente** una volta terminato l'utilizzo.

Il programma di seguito dimostra proprio questa situazione problematica: viene utilizzata una **variabile automatica**, ovvero allocata nello stack, come valore restituito da `**pthread_exit**`. Di conseguenza, il puntatore che viene recuperato tramite `**pthread_join**` può riferirsi a memoria già riutilizzata o non più accessibile, portando a risultati imprevisti o a errori di esecuzione.

Esempio

Questo esempio mette in luce un **problema critico** nella gestione della memoria quando si usano thread in C con la libreria POSIX (`pthread`). L'obiettivo è mostrare cosa accade quando un thread restituisce un **puntatore a una struttura locale allocata sullo stack**, e come ciò possa portare a risultati imprevedibili e non portabili tra sistemi.

```
void *
thr_fn1(void *arg)
{
    struct foo    foo = {1, 2, 3, 4}; //sullo stack

    printf("thread 1:\n", &foo);
    pthread_exit((void *)&foo);
}
void *
thr_fn2(void *arg)
{
    printf("thread 2: ID is %lu\n", (unsigned long)pthread_self());
    pthread_exit((void *)0);
}
```


Nel codice presentato, la struttura `foo` viene definita come contenente quattro interi. Nella funzione `thr_fn1`, questa struttura viene **allocata sullo stack** e inizializzata con i valori 1, 2, 3 e 4. Successivamente, il thread stampa i contenuti della struttura e li restituisce tramite `pthread_exit`, passando il suo indirizzo.

```
int main(void)
{
    int                err;

    pthread_t          tid1, tid2;
    struct foo         *fp;

    err = pthread_create(&tid1, NULL, thr_fn1, NULL);
    if (err != 0):
        {printf("can't create thread 1: %s\n", strerror(err)); exit(1);}

    err = pthread_join(tid1, (void *)&fp);
    if (err != 0)
        {printf("can't join with thread 1: %s\n",strerror(err)); exit(1);}

    sleep(1);
    printf("parent starting second thread\n");
    err = pthread_create(&tid2, NULL, thr_fn2, NULL);
    if (err != 0)
        {printf("can't create thread 2: %s\n",strerror(err));exit(1);}

    sleep(1);
    printf("parent:\n", fp);
    exit(0);
}
```

Il thread principale (`main`) riceve il puntatore a questa struttura tramite `pthread_join` e lo assegna a un puntatore `fp`. Dopo una breve pausa (`sleep(1)`), crea un secondo thread (`thr_fn2`) e, successivamente, stampa di nuovo i valori della struttura attraverso il puntatore `fp`.

Il problema nasce proprio qui. Poiché la struttura è stata creata sullo stack del primo thread (`thr_fn1`), una volta che questo thread termina, la memoria associata al suo stack può essere **riciclata** dal sistema. Di conseguenza, quando il `main` accede alla struttura, potrebbe trovarsi davanti a **valori corrotti o non validi**, oppure a un **segment fault** nei casi peggiori.

I risultati dell'esecuzione su diversi sistemi operativi lo confermano. Su Linux, si osserva che l'indirizzo della struttura è riutilizzato, ma i dati sono alterati. Su Solaris, i valori risultano corrotti. Su macOS, si verifica un **crash con segment fault**, mentre su FreeBSD il comportamento è corretto, ma non c'è alcuna garanzia che ciò avvenga sempre.

Questo comportamento mette in evidenza che **la memoria stack di un thread non è affidabile dopo che il thread è terminato**. Il suo contenuto può essere sovrascritto da altri thread, specialmente se vengono creati nuovi thread poco dopo la sua terminazione.

Per prevenire questo tipo di errore, è consigliabile **non restituire mai** puntatori a **variabili locali** di un thread. Le alternative corrette sono:

- Allocare dinamicamente la struttura con `**malloc**`, in modo che la sua vita non sia legata allo stack del thread.
- Usare una **struttura globale** condivisa, se il contenuto è destinato ad essere accessibile da più thread.
- Copiare il contenuto necessario prima che il thread venga chiuso.

Cancellazione di thread

Un **thread** può richiedere che un altro thread, appartenente allo stesso processo, venga terminato utilizzando la funzione `pthread_cancel`. La firma di questa funzione è la seguente:

```
#include <pthread.h>
int pthread_cancel(pthread_t tid);
//restituisce 0 se OK, numero di errore se fallisce
```

Il parametro `tid` rappresenta l'identificatore del thread da cancellare. La funzione restituisce **0 in caso di successo**, oppure un **codice di errore** se la richiesta fallisce.

Quando viene invocata, `pthread_cancel` **non interrompe immediatamente** l'esecuzione del thread, ma invia semplicemente una **richiesta di cancellazione**. Il thread specificato **potrà essere cancellato solo se è configurato per accettare cancellazioni** e si trova in uno **stato annullabile**. Per impostazione predefinita, quando un thread viene cancellato, il suo comportamento è equivalente a una chiamata a `pthread_exit` con valore di ritorno `PTHREAD_CANCELED`.

Va sottolineato che `pthread_cancel` **non blocca l'esecuzione del thread chiamante**: non aspetta che il thread specificato termini realmente, ma si limita a **segnalare** la cancellazione.

Per quanto riguarda la gestione della terminazione, lo **stato di uscita di un thread viene mantenuto in memoria** fino a quando il thread principale o un altro thread non invoca `pthread_join` su quel thread. Questo meccanismo consente al chiamante di recuperare lo stato di terminazione, compresi eventuali valori restituiti o codici di cancellazione.

Tuttavia, se un thread è stato **marcato come "detached"** (staccato), allora il sistema può **rilasciare immediatamente la memoria associata** a quel thread non appena termina. In questa configurazione, non è più possibile utilizzare `pthread_join`, poiché **un thread staccato non può essere raggiunto o monitorato** tramite quella funzione.

Questo significa che l'uso combinato di `pthread_cancel`, `pthread_join` e lo stato **detach** dei thread deve essere pianificato attentamente per evitare **perdite di risorse, accessi a memoria invalidi o comportamenti non deterministici**.

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void* thread_lento(void* arg) {
    printf("Thread avviato, entro in ciclo infinito...\n");
    while (1) {
        printf("Thread attivo...\n");
        sleep(1); // Simula lavoro
    }
    return NULL;
}
```

```
int main() {
    pthread_t tid;
    void* result;

    // Crea il thread
    if (pthread_create(&tid, NULL, thread_lento, NULL) != 0) {
        perror("Errore creazione thread");
        return 1;
    }

    // Attende un po' e poi cancella il thread
    sleep(3);
    printf("Main: invio richiesta di cancellazione al thread\n");
    if (pthread_cancel(tid) != 0) {
        perror("Errore cancellazione thread");
        return 1;
    }

    // Attende la terminazione del thread
    if (pthread_join(tid, &result) != 0) {
        perror("Errore join thread");
        return 1;
    }
}
```

```

// Attende la terminazione del thread
if (pthread_join(tid, &result) != 0) {
    perror("Errore join thread");
    return 1;
}

// Verifica se il thread è stato cancellato
if (result == PTHREAD_CANCELED) {
    printf("Main: il thread è stato cancellato con successo.\n");
} else {
    printf("Main: il thread ha terminato normalmente.\n");
}

return 0;
}

```

- Viene creato un thread (`thread_lento`) che entra in un ciclo infinito.
- Il `main()` attende 3 secondi (`sleep(3)`), poi invia una **richiesta di cancellazione** con `pthread_cancel`.
- Il `main()` attende che il thread termini con `pthread_join`.
- Infine, controlla se il valore di ritorno è `PTHREAD_CANCELED`, confermando l'avvenuta cancellazione.

NB: Perché la cancellazione funzioni, il thread **deve trovarsi in uno stato cancellabile**. Il ciclo infinito funziona perché `sleep()` è un **punto di cancellazione**: è una funzione che consente l'interruzione del thread.

Distacco di un thread

```

#include <pthread.h>
int pthread_detach(pthread_t tid);
//restituisce 0 se OK, numero di errore se fallisce

```

Nel modello **POSIX dei thread**, ogni thread può assumere uno dei due stati fondamentali relativi alla gestione del proprio ciclo di vita: **joinable** oppure **detached**. Per **default**, un thread è **joinable**, ovvero può essere **atteso** tramite la funzione `pthread_join`, che permette anche di **recuperare il valore di ritorno** del thread al termine della sua esecuzione. In alternativa, un thread può essere creato o convertito in stato **detached**, cioè completamente **autonomo nella gestione delle proprie risorse**, che vengono liberate automaticamente dal sistema operativo al termine dell'esecuzione, senza necessità di sincronizzazione esplicita.

Il passaggio allo stato **detached** può avvenire in due modalità:

- La prima consiste nell'invocare la funzione `pthread_detach`, che riceve come argomento l'identificatore del thread da distaccare e restituisce **0 in caso di successo**, o un **codice di errore** in caso contrario. Una volta che un thread è stato distaccato, **non è più possibile invocare `pthread_join`** su di esso: tale chiamata fallisce con l'errore `EINVAL`, poiché il thread non conserva più il proprio **stato di uscita**. Questo approccio è utile quando si desidera creare thread **usa e getta**, cioè non soggetti a sincronizzazione o controllo esplicito al termine.
- La seconda modalità prevede di configurare **fin dalla creazione** lo stato di detach. Ciò avviene tramite l'uso di una struttura di tipo `pthread_attr_t`, che permette di definire gli attributi del thread prima della sua attivazione con `pthread_create`.

Attributi dei thread

```
include <pthread.h>

int pthread_attr_init(pthread_attr_t *attr);
```

Nel modello POSIX, gli attributi di un thread sono rappresentati attraverso la struttura `pthread_attr_t`, la cui implementazione è **opaca**, ossia non accessibile direttamente dal programmatore. Per modificare o accedere agli attributi si utilizzano apposite **funzioni dedicate**. Tra gli attributi principali si distinguono:

- **scope**, che definisce l'ambito di competizione per le **risorse di CPU**. I valori possibili sono:
 - `PTHREAD_SCOPE_PROCESS` (predefinito): competizione limitata ai thread del processo corrente;
 - `PTHREAD_SCOPE_SYSTEM`: competizione estesa a tutti i thread del sistema, con scheduling gestito dal kernel.
- **detachstate**, che specifica se il thread sarà **joinable** o **detached**. Il valore predefinito è `PTHREAD_CREATE_JOINABLE`, che consente l'attesa tramite `pthread_join`. In alternativa, si può impostare `PTHREAD_CREATE_DETACHED`, rendendo il thread autonomo nella liberazione delle risorse.
- **stackaddr**, che consente di assegnare un **indirizzo personalizzato per lo stack** del thread. Il valore predefinito è `NULL`, considerato deprecato e gestito automaticamente dal sistema.
- **stacksize**, che definisce la **dimensione dello stack**. Il valore predefinito è `NULL`, ma può essere impostato esplicitamente, purché non sia inferiore a `PTHREAD_STACK_MIN`.
- **schedparam**, che stabilisce la **priorità** del thread tramite una struttura contenente un intero. I valori variano da **1 a 99** per le politiche con priorità, mentre il valore **0** viene usato per le politiche senza priorità.
- **schedpolicy**, che determina la **politica di scheduling** del thread. Le opzioni includono:
 - `SCHED_OTHER` (default): gestione tramite time-slice;
 - `SCHED_FIFO`: priorità in ordine di arrivo;
 - `SCHED_RR`: round robin con priorità.
- **inheritsched**, che indica se usare la politica e la priorità del **thread chiamante** (`PTHREAD_INHERIT_SCHED`, valore predefinito) o quelle **esplicitamente definite** negli attributi (`PTHREAD_EXPLICIT_SCHED`).

Per inizializzare e distruggere un oggetto `pthread_attr_t`, si utilizzano rispettivamente le funzioni:

```
#include <pthread.h>
int pthread_attr_init (pthread_attr_t *attr);
int pthread_attr_destroy (pthread_attr_t *attr);
```

Entrambe restituiscono **0 in caso di successo**, oppure un **codice di errore**.

Per **ottenere o impostare** i valori dei singoli attributi, si usano le seguenti funzioni, sostituendo `attribute` con il nome specifico dell'attributo:

```
#include <pthread.h>
int pthread_attr_get<attribute> (const pthread_attr_t *attr,
*attribute);
int pthread_attr_set<attribute> (const pthread_attr_t *attr,
attribute);
```

Tali funzioni restituiscono **0** in caso di successo o un altro valore in caso di errore. Gli attributi devono essere **inizializzati** prima dell'uso e **distrutti** al termine per liberare eventuali risorse.

Thread vs Processo

Primitiva di processo	Primitiva di thread	Descrizione
<code>fork</code>	<code>pthread_create</code>	Crea un nuovo flusso di controllo
<code>exit</code>	<code>pthread_exit</code>	Esce da un flusso di controllo esistente
<code>waitpid</code>	<code>pthread_join</code>	Acquisisce lo stato di uscita dal flusso di controllo
<code>getpid</code>	<code>pthread_self</code>	Determina l'id del flusso di controllo
<code>abort</code>	<code>pthread_cancel</code>	Richiede la terminazione anomala del flusso di controllo

Esempio

```
#include "apue.h"
#include <pthread.h>
int
makethread(void *(*fn)(void *), void *arg)
{
    int err;
    pthread_t tid;
    pthread_attr_t attr;

    err = pthread_attr_init(&attr);
    if (err != 0)
        return(err);
    err = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    if (err == 0)
        err = pthread_create(&tid, &attr, fn, arg);
    pthread_attr_destroy(&attr);
    return(err);
}
```

La funzione **`**makethread**`** è una funzione generica che accetta due parametri:

- un **puntatore a funzione** `fn`, cioè la funzione che il thread dovrà eseguire,
- un **argomento** `arg` da passare a quella funzione.

La funzione ha lo scopo di creare un nuovo thread **già distaccato** al momento della creazione.

All'interno della funzione, viene dichiarata una variabile `pthread_attr_t attr`, che rappresenta la struttura di attributi del thread. Viene inizializzata con **`pthread_attr_init`**. Se l'inizializzazione fallisce, la funzione termina restituendo il codice di errore.

Successivamente, viene impostato l'attributo `detachstate` con **`pthread_attr_setdetachstate`**, assegnandogli il valore **`PTHREAD_CREATE_DETACHED`**. Questo indica che il thread sarà creato in modalità **detached**, e quindi non sarà necessario (né possibile) usare `pthread_join` su di esso.

Se anche questa operazione va a buon fine, viene creato effettivamente il thread tramite `pthread_create`, passando gli attributi modificati. Infine, la struttura `attr` viene **distrutta** con `pthread_attr_destroy`, in modo da liberare risorse.

La funzione restituisce il valore di ritorno di `pthread_create` oppure il primo errore che si è verificato, consentendo al chiamante di gestire eventuali problemi.

Questo codice è utile quando si vogliono **creare thread che non devono essere mai sincronizzati** con il chiamante, ad esempio per attività secondarie, servizi autonomi o operazioni “usa e getta”, migliorando la **gestione automatica delle risorse** da parte del sistema.

Rientranza

In ambiente concorrente, una funzione è detta **rientrante** se può essere **interrotta a metà della sua esecuzione** e poi **richiamata di nuovo** (dallo stesso thread o da un altro) senza compromettere la correttezza del programma.

I **thread** condividono con i **gestori di segnali** una caratteristica importante: entrambi possono **attivare l'esecuzione concorrente** della stessa funzione. Ad esempio, se più thread chiamano simultaneamente la stessa funzione, è possibile che essa venga eseguita **più volte contemporaneamente**.

In questi casi, è fondamentale che la funzione sia **thread-safe**, cioè in grado di gestire queste chiamate multiple **in modo sicuro**, senza conflitti di dati o comportamenti imprevisti.

Le implementazioni POSIX che supportano funzioni **sicure per i thread** definiscono il simbolo `_POSIX_THREAD_SAFE_FUNCTIONS` nell'header `<unistd.h>`. Questo simbolo indica che sono disponibili **varianti rientranti** di molte funzioni standard della libreria C (ad esempio `strtok_r`, `asctime_r`, `ctime_r`).

È inoltre possibile, a **runtime**, verificare se il sistema supporta funzioni thread-safe usando la funzione `sysconf`, con il parametro `_SC_THREAD_SAFE_FUNCTIONS`.

Thread e segnali

Maschera e disposizione dei segnali

Ogni **thread** in un processo POSIX dispone di una **maschera di segnali propria**, ovvero può **bloccare o sbloccare** segnali in modo indipendente dagli altri thread. Questo permette a ciascun thread di decidere **quali segnali accettare e quali ignorare**.

Tuttavia, la **disposizione del segnale** (cioè l'azione da compiere al verificarsi del segnale: ignorare, eseguire un gestore, terminare, ecc.) è **condivisa tra tutti i thread** del processo. Quindi, se un thread modifica l'azione associata a un segnale (ad esempio installando un nuovo gestore), questa modifica ha effetto **su tutti i thread**.

Questo comporta che se un thread sceglie di **ignorare un segnale**, un altro thread può **ripristinarne l'azione di default** oppure **installare un proprio gestore**, influenzando l'intero processo.

Consegna e invio dei segnali

I **segnali** vengono **consegnati a un solo thread per processo**, non a tutti. Se il segnale deriva da un **errore hardware** (ad esempio divisione per zero), viene tipicamente recapitato **al thread che ha generato l'evento**. Per altri tipi di segnali, il thread destinatario viene scelto **in modo arbitrario** dal sistema.

Per inviare un segnale a **un processo intero**, si utilizza la funzione `kill(pid, signo)`. Se invece si vuole inviare un segnale a **uno specifico thread**, si usa la funzione `pthread_kill`:


```
#include <signal.h>

int pthread_kill (pthread_t thread, int signo);

//restituisce 0 se OK, numero di errore se fallisce
```

Questa funzione restituisce **0 se ha successo**, o un **codice di errore** in caso di fallimento. Se `signo` è uguale a 0, la funzione non invia nessun segnale ma **verifica soltanto l'esistenza del thread**.

È importante ricordare che se l'azione di default per il segnale specificato è la **terminazione del processo**, allora l'invio del segnale a un singolo thread causerà comunque la **terminazione dell'intero processo**.

La chiamata clone di LINUX

Il **kernel di Linux** utilizza la chiamata di sistema **clone** per creare nuovi processi o, più precisamente, per controllare con precisione **quali risorse devono essere condivise** tra il processo genitore e quello figlio. A differenza di `fork`, che crea un processo figlio con una copia quasi completa del contesto del genitore, `clone` permette di specificare cosa **condividere** (memoria, file, segnali, PID, ecc.) e cosa **duplicare**.

La funzione `clone` ha questa firma:

```
int clone(int (*fn) (void *), void *child_stack, int flags, void *arg);
```

Questa funzione crea un nuovo processo o thread e inizia l'esecuzione della funzione `fn`, passando come parametro `arg`, e utilizzando `child_stack` come stack per il nuovo contesto di esecuzione.

Il comportamento di `clone` dipende dai **flag** specificati, che controllano il grado di condivisione tra i due contesti:

Flag	Se settato	Se non settato
<code>CLONE_VM</code>	Condivide la memoria virtuale (crea un thread)	Crea un processo con spazio di memoria separato
<code>CLONE_FS</code>	Condivide le impostazioni del file system (umask, cwd)	Mantiene impostazioni distinte
<code>CLONE_FILES</code>	Condivide i file descriptor	I file descriptor vengono copiati
<code>CLONE_SIGHAND</code>	Condivide la tabella dei gestori di segnali	Ogni processo ha la propria tabella
<code>CLONE_PID</code>	Il figlio mantiene lo stesso PID (per thread kernel)	Il figlio riceve un nuovo PID

Un programma può invocare direttamente `clone`, ma in tal caso diventa **non portabile**: il codice sarà valido solo su Linux, poiché `clone` **non fa parte dello standard POSIX**. Proprio per questo motivo, le librerie di threading (come `pthread`) **utilizzano `clone` internamente**, ma offrono un'interfaccia standardizzata e portabile all'esterno.

Libreria Pthread

Verifica del supporto dei thread

Il supporto ai thread POSIX può essere verificato controllando la presenza della macro `_POSIX_THREADS`, definita nel file header `<unistd.h>`. Inoltre, è possibile usare la funzione `sysconf` con l'argomento `_SC_THREADS` per verificare dinamicamente se i thread sono supportati dal sistema:

```
long sysconf (int name)
```

Se il valore restituito è `0`, i thread sono supportati; se è `-1`, non lo sono.

Numero massimo di thread

Esiste un **numero massimo di thread** che un processo può creare. Questo valore è rappresentato dalla macro `PTHREAD_THREADS_MAX`, definita in `<limits.h>`. È anche possibile ottenerlo a **runtime** tramite `sysconf`, usando il parametro `_SC_THREAD_THREADS_MAX`.

Compilazione con pthread

Per utilizzare i thread POSIX in un programma, è necessario:

1. **Includere** l'header `<pthread.h>`.
2. **Linkare esplicitamente** la libreria `pthread` durante la compilazione, utilizzando l'opzione `** -lpthread**`.

Esempio di comando:

```
gcc programma.c -lpthread
```

Esempio 1

Questo esempio mostra come creare e gestire **thread indipendenti** in un programma C utilizzando la **libreria POSIX Pthread**. L'obiettivo è avviare due thread separati, ognuno incaricato di stampare un messaggio, e garantire che il programma principale attenda la fine della loro esecuzione prima di terminare.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void *print_message_function( void *ptr );
int main()
{
    pthread_t thread1, thread2;
    char *message1 = "Thread 1";
    char *message2 = "Thread 2";
    int  iret1, iret2;
    /* crea thread indipendenti, ciascuno dei quali eseguirà una funzione */
    iret1 = pthread_create(&thread1, NULL, print_message_function, (void*)message1);
    iret2 = pthread_create(&thread2, NULL, print_message_function, (void*) message2);

    /*aspetta che i thread abbiano completato prima che il main continui. Se non aspettiamo si
    potrebbe eseguire una exit che terminerà l'intero processo e tutti i suoi thread, prima che
    questi abbiano finito */

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Thread 1 returns: %d\n",iret1);
    printf("Thread 2 returns: %d\n",iret2);
    exit(0);
}
void *print_message_function( void *ptr )
{
    char *message;
    message = (char *) ptr;
    printf("%s \n", message);
}
```

Nel corpo della funzione `main()`, vengono dichiarati due identificatori di thread di tipo `pthread_t`, due stringhe contenenti i messaggi da stampare e due interi per conservare il risultato delle chiamate a `pthread_create`. I thread vengono creati con la funzione `pthread_create`, specificando come funzione da eseguire `print_message_function` e passando come argomento uno dei due messaggi. In questo modo, ogni thread riceve un parametro diverso che rappresenta il messaggio da visualizzare.

Affinché il programma principale attenda che i thread terminino correttamente, vengono utilizzate due chiamate a `pthread_join`, una per ciascun thread. Queste chiamate sono essenziali: senza di esse, il **processo principale** potrebbe terminare prima che i thread completino l'esecuzione, causando l'interruzione forzata dei thread stessi. In altre parole, `pthread_join` blocca l'esecuzione del `main` finché il thread associato non ha concluso il suo compito.

La funzione `print_message_function` riceve un argomento generico (`void *`) che rappresenta un puntatore al messaggio da stampare. Questo puntatore viene convertito in un `char *` e passato a `printf`, permettendo di visualizzare il contenuto del messaggio sulla console. In questo modo, ogni thread esegue in parallelo la stampa della stringa ricevuta.

Infine, dopo l'attesa con `pthread_join`, il programma stampa i codici di ritorno ottenuti da `pthread_create` e termina con una chiamata a `exit(0)`, indicando una terminazione corretta del processo. Questo esempio dimostra in modo chiaro come utilizzare i thread per eseguire operazioni concorrenti e come garantire che il programma attenda correttamente la conclusione di tutti i thread prima di terminare.

Esempio 2

Questo secondo esempio illustra come avviare più thread in un ciclo, assegnando a ciascuno un identificatore univoco. L'obiettivo è stampare, da ciascun thread, il proprio **ID**, ricevuto come parametro. L'esempio è particolarmente utile per comprendere un errore comune che riguarda il **passaggio di argomenti a thread concorrenti** e come risolverlo in modo sicuro.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_THREADS 8
/* Questa routine riceve in ingresso l'id del thread */
void *PrintHello(void *threadid)
{
    int *id_ptr, taskid;

    sleep(1);
    id_ptr = (int *) threadid;
    taskid = *id_ptr;
    printf("Thread %d\n", taskid);
    pthread_exit(NULL);
}
```

Nella funzione `PrintHello`, ogni thread riceve un **argomento generico**, un `void *`, che rappresenta l'indirizzo dell'ID del thread. Tale puntatore viene convertito in un `int *` e dereferenziato per ottenere il valore da stampare. Il ritardo introdotto con la chiamata `sleep(1)` serve a simulare una condizione in cui l'esecuzione del thread è **ritardata rispetto alla creazione**, aumentando la probabilità che si verifichi il problema evidenziato.

Errato:

```
int main(int argc, char *argv[])
{
pthread_t threads[NUM_THREADS];
int rc, t;
for(t=0;t<NUM_THREADS;t++) {
    printf("Creating thread %d\n", t);
    rc = pthread_create(&threads[t], NULL, PrintHello, (void*)&t);
    if (rc) {
        printf("ERROR; return code from pthread_create() is %d\n", rc);
        exit(-1);
    }
}
pthread_exit(NULL);
}
```

Nel primo frammento, marcato come errato, all'interno del ciclo viene passato direttamente **l'indirizzo della variabile `t`**, cioè `(void *)&t`, a tutti i thread. Poiché `t` è una variabile **locale alla funzione `main`**, essa è allocata **sullo stack** e viene **riutilizzata a ogni iterazione del ciclo**. Di conseguenza, tutti i thread ricevono **lo stesso indirizzo di memoria**. Poiché i thread iniziano la loro esecuzione in tempi diversi (anche a causa del `sleep`), quando ciascuno di essi accede a quell'indirizzo, il contenuto può essere **già stato modificato** dalle iterazioni successive. Questo porta a un comportamento **non deterministico e scorretto**, in cui i thread stampano valori errati o ripetuti.

```
int main(int argc, char *argv[])
{
pthread_t threads[NUM_THREADS];
int *taskids[NUM_THREADS];
int rc, t;
for(t=0;t<NUM_THREADS;t++) {
    taskids[t] = (int *)malloc(sizeof(int)); /* indirizzo di un intero */
    *taskids[t] = t;                        /* l'intero */
    printf("Creating thread %d\n", t);
    rc = pthread_create(&threads[t], NULL, PrintHello, (void *) taskids[t]);
    if (rc) {
        printf("ERROR; return code from pthread_create() is %d\n", rc);
        exit(-1);
    }
}
pthread_exit(NULL);
}
```

Nel secondo frammento, corretto, si evita questo problema allocando dinamicamente la memoria con `malloc`. Per ogni iterazione del ciclo, viene riservato un **blocco di memoria sull'heap**, che non viene sovrascritto automaticamente come avviene nello stack. A ogni thread viene passato **un puntatore a una copia distinta** del valore `t`. L'heap, a differenza dello stack, mantiene stabile il contenuto allocato finché

non viene esplicitamente liberato, garantendo che ogni thread acceda a **un proprio ID coerente e immutabile**.

Inoltre, alla fine del `main` viene invocata la funzione `pthread_exit(NULL)`, che garantisce che il **thread principale non termini prematuramente**, permettendo a tutti i thread figli di completare la propria esecuzione.

Esempio di esecuzione

Caso **errato** – uso della variabile `t` sullo **stack** (stesso indirizzo per tutti)

Codice coinvolto (semplificato):

```
for (t = 0; t < NUM_THREADS; t++) {  
    pthread_create(&threads[t], NULL, PrintHello, (void *)&t);  
}
```

Esempio di output:

```
Creating thread 0  
Creating thread 1  
Creating thread 2  
Creating thread 3  
Creating thread 4  
Creating thread 5  
Creating thread 6  
Creating thread 7  
Thread 8  
Thread 8  
Thread 8  
Thread 8  
Thread 8  
Thread 8  
Thread 8  
Thread 8
```

Cosa succede?

La variabile `t` assume progressivamente i valori da 0 a 7, ma i thread **non partono immediatamente**. Quando si svegliano dopo `sleep(1)`, trovano che `t` ha già raggiunto il valore finale (8), che quindi viene **letto da tutti**, perché accedono allo **stesso indirizzo di memoria sullo stack**.

Caso **corretto** – uso dell'**heap** con `malloc`

Codice coinvolto (semplificato):

```
taskids[t] = malloc(sizeof(int));  
*taskids[t] = t;  
pthread_create(&threads[t], NULL, PrintHello, (void *)taskids[t]);
```

Esempio di output:

```
Creating thread 0  
Creating thread 1  
Creating thread 2  
Creating thread 3  
Creating thread 4  
Creating thread 5  
Creating thread 6  
Creating thread 7  
Thread 1  
Thread 3  
Thread 0  
Thread 2  
Thread 5  
Thread 4  
Thread 6  
Thread 7
```

Cosa succede?

Ogni thread riceve un **puntatore distinto** a un intero allocato sull'**heap**, contenente il proprio ID. Anche se i thread si risvegliano in ordine **casuale** (a causa della concorrenza e del sistema operativo), ognuno stampa **il numero corretto**.

L'ordine dell'output può variare a ogni esecuzione, ma **i numeri saranno corretti e unici**.

Capitolo 17 – Mutex e Variabili di Condizione

Mutex e Variabili di Condizione

Nel contesto della programmazione concorrente, quando più **thread** condividono la stessa **memoria**, è essenziale mantenere la **coerenza dei dati**. Se ciascun **thread** utilizza solo variabili **private**, non sorgono problemi. Tuttavia, l'accesso concorrente a **variabili condivise** può introdurre **inconsistenze**. Un esempio classico si verifica quando un **thread** scrive su una variabile in più fasi mentre un altro la legge simultaneamente, producendo così un **valore corrotto** o **parziale**.

Per prevenire tali problematiche, è necessario **sincronizzare** i **thread** tramite un **meccanismo di lock**, che consente a un solo **thread** per volta di accedere alla risorsa condivisa. Quando un **thread** desidera eseguire un'operazione, deve **acquisire** il **lock**; al termine, rilasciarlo, consentendo ad altri **thread** l'accesso alla stessa risorsa.

Uno scenario critico emerge quando più **thread** modificano **contemporaneamente** una variabile. Operazioni apparentemente semplici, come l'**incremento**, richiedono più fasi: lettura, modifica e scrittura. Se queste fasi non sono **sincronizzate**, i risultati possono essere **errati**, sovrascrivendo dati di altri **thread**.

Tali problemi possono essere evitati rendendo le operazioni **atomiche**, ovvero indivisibili, eliminando così la possibilità di **race condition**. Quando le operazioni sono **sequenzialmente consistenti**, non sono necessari meccanismi di sincronizzazione aggiuntivi, poiché l'esecuzione appare **coerente** per tutti i **thread**. Tuttavia, in **architetture multicore** o **multiprocessore**, l'ordine di esecuzione non è garantito, rendendo indispensabile l'uso di **meccanismi espliciti di sincronizzazione**.

Lo standard **POSIX** definisce diversi strumenti per la **corretta sincronizzazione** dei **thread**, principalmente tramite **primitive** come i **mutex** e le **variabili di condizione**. Lo standard **POSIX.1c** prevede che i **mutex** siano usati per garantire la **mutua esclusione**, mentre le **variabili di condizione** permettono di **attendere** o **segnalare eventi** tra **thread**. Lo standard **POSIX.1b**, inoltre, include i **semafori** per gestire la **concorrenza** anche tra **processi distinti**.

Mutex

Un **mutex** (definito in *POSIX.1c*) è un oggetto che permette a processi o thread concorrenti di **sincronizzare l'accesso ai dati condivisi** in modo tale da evitare **race condition**. Il mutex può essere **bloccato** oppure **non bloccato** e in quest'ultimo caso i thread che tentano di bloccarlo nuovamente (devono accedere in sezione critica) restano **in attesa**, venendo sbloccati (uno solo) quando il mutex viene rilasciato. In C il mutex è rappresentato dal tipo `pthread_mutex_t` e può essere **inizializzato** in modo **statico** assegnandogli direttamente il valore `PTHREAD_MUTEX_INITIALIZER` oppure in modo **dinamico**.

```
#include <pthread.h>
int pthread_mutex_init (pthread_mutex_t *mutex, const
pthread_mutexattr_t *attr);
int pthread_mutex_destroy (pthread_mutex_t *mutex);
```

Queste funzioni servono rispettivamente per **inizializzare un mutex dinamico** e per **distruggerlo**.

Restituiscono 0 in caso di successo, altrimenti un numero diverso. L'argomento `mutex` è il puntatore al mutex. L'argomento `attr` è la struttura dati che rappresenta gli **attributi** del mutex.


```
#include <pthread.h>
int pthread_mutexattr_init (pthread_mutexattr_t *attr);
int pthread_mutexattr_destroy (pthread_mutexattr_t *attr);
```

Queste funzioni servono rispettivamente per **inizializzare la struttura degli attributi** del mutex e per **distruggerla**. Restituiscono **0** in caso di successi, altrimenti un numero diverso. L'argomento `attr` è la struttura degli attributi.

```
#include <pthread.h>
int pthread_mutexattr_set<attribute> (pthread_mutexattr_t
*attr, int attrVal);
int pthread_mutexattr_get<attribute> (pthread_mutexattr_t
*attr, int *attrVal);
```

Queste funzioni servono rispettivamente per **impostare o eliminare un attributo** da una specifica struttura degli attributi. Restituiscono **0** in caso di successo, altrimenti un numero diverso. L'argomento `attr` è la struttura degli attributi. L'argomento `attrVal` è la variabile che contiene (o conterrà) il valore dell'attributo `<attribute>`. Un esempio è l'attributo **pshared** che può contenere il valore `PTHREAD_PROCESS_SHARED` (condivide il mutex con tutti i thread del processo) oppure il valore `PTHREAD_PROCESS_PRIVATE` (l'opposto).

```
#include <pthread.h>
int pthread_mutex_lock (pthread_mutex_t *mutex);
int pthread_mutex_unlock (pthread_mutex_t *mutex);
int pthread_mutex_trylock (pthread_mutex_t *mutex);
```

Queste funzioni servono per **interagire con il mutex**. Restituiscono **0** in caso di successo oppure un numero se falliscono (flag `<errno.h>`). L'argomento `mutex` corrisponde al mutex inizializzato in precedenza.

La funzione `pthread_mutex_lock()` è **bloccante**, serve per **bloccare il mutex** e ritorna solo se è stato bloccato, ponendo nel mentre il thread chiamante in attesa.

La funzione `pthread_mutex_unlock()` serve per **rilasciare un mutex**, **risvegliando** uno dei thread in attesa scelto dalla politica di scheduling.

La funzione `pthread_mutex_trylock()` è **non bloccante**, serve per **bloccare il mutex** restituendo l'errore `EBUSY` se questo è già bloccato da un altro thread (quindi non mette in attesa come `pthread_mutex_lock()`).

Come si usa

1. **Creare il mutex.** Per un **mutex statico** bisogna dichiarare una semplice variabile `pthread_mutex_t` alla quale gli si associa il valore `PTHREAD_MUTEX_INITIALIZER`.

Per un **mutex dinamico** bisogna: dichiarare un puntatore di tipo `pthread_mutex_t`, associargli memoria con `malloc()` e infine invocare `pthread_mutex_init()` (il secondo argomento può essere `NULL` se si vuole inizializzare il mutex con le impostazioni di **default**, cioè condiviso tra tutti i thread).

2. **Interagire con il mutex** attraverso `pthread_mutex_lock()` se si vuole bloccare (**entrare in sezione critica**) e `pthread_mutex_unlock()` se si vuole sbloccare (**uscire dalla sezione critica**).
3. **Eliminare il mutex solo se è dinamico**, quindi invocare `pthread_mutex_destroy()`. Il mutex statico è una semplice variabile locale, viene deallocata automaticamente quando termina il processo.

Esempio Race Condition

```
//Interferenza...include omissi...

void * thread_function(void *);

int myglobal; // variabile globale int

int main(void) {
    pthread_t mythread;
    int i;
    if(pthread_create(&mythread,NULL,thread_function,NULL)){
        printf("creazione del thread fallita.\n");
        exit(1);
    }
    for (i=0; i<20; i++) {
        myglobal=myglobal+1;
        printf("o");
        fflush(stdout);
        sleep(1);
    }
    if (pthread_join(mythread,NULL)) {
        printf("errore nel join dei thread.");
        exit(2);
    }
    printf("\nmyglobal e' uguale a %d\n",myglobal);
    exit(0);}
```

```

...
void *thread_function(void *arg) {
    int i,j;
    for (i=0; i<20;i++) {
        j=myglobal;
        j=j+1;
        printf(".");
        fflush(stdout);
        sleep(1);
        myglobal=j;
    }
    return NULL;
}

```

Nel programma presentato, due thread (il **main thread** e uno secondario creato con `pthread_create`) operano su una **variabile globale condivisa** chiamata `myglobal`. Entrambi i thread incrementano il valore della variabile all'interno di un ciclo ripetuto 20 volte.

Il **main thread** esegue direttamente `myglobal = myglobal + 1`, mentre il thread secondario lo fa in tre passi separati (lettura, incremento e scrittura), simulando un'operazione non atomica. Durante l'esecuzione, non esiste **sincronizzazione** tra i thread, il che porta a una **race condition**: entrambi i thread leggono e scrivono `myglobal` senza alcuna protezione.

Questo può causare **comportamenti non deterministici**, come mostrato nel possibile output. Ad esempio, anche se ogni thread esegue 20 incrementi, il risultato finale di `myglobal` può essere **inferiore a 40**, come nel caso dell'output `myglobal` è uguale a 21.

Questo comportamento errato è dovuto al fatto che più thread possono leggere lo stesso valore di `myglobal` prima che uno dei due completi la scrittura. Quando entrambi lo aggiornano basandosi su una **vecchia copia**, uno dei due aggiornamenti viene **perso**.

Questo esempio illustra chiaramente la necessità di **sincronizzare l'accesso** a variabili condivise per evitare **corse critiche**. In mancanza di meccanismi come i **lock**, anche un semplice incremento può produrre **risultati imprevedibili**.

Esempio (no race)

```
void * thread_function(void *);
int myglobal;
pthread_mutex_t mymutex=PTHREAD_MUTEX_INITIALIZER;

int main(void) {
    pthread_t mythread;
    int i;
    if (pthread_create(&mythread,NULL,thread_function,NULL)) {printf("creazione del thread fallita.");exit(1);}
    for (i=0; i<20; i++) {
        pthread_mutex_lock(&mutex);
        myglobal = myglobal+1;
        pthread_mutex_unlock(&mutex);
        printf("o");fflush(stdout);
        sleep(1);
    }
    if(pthread_join(mythread,NULL)) {
        printf("errore nel join con il thread.\n");
        exit(2);
    }
    printf("\nmyglobal è uguale a %d\n",myglobal);
    exit(0);}...

void *thread_function(void *arg) {
    int i,j;
    for ( i=0; i<20; i++ ) {
        pthread_mutex_lock(&mutex);
        j=myglobal;
        j=j+1;
        printf(".");
        fflush(stdout);
        sleep(1);
        myglobal=j;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Il programma mostra un esempio corretto di accesso concorrente a una variabile globale condivisa tra due **thread**, in cui si evita la **race condition** grazie all'uso di un **mutex**.

La variabile `myglobal` viene protetta mediante un mutex (`pthread_mutex_t mymutex`), inizializzato con `PTHREAD_MUTEX_INITIALIZER`. Ogni volta che un thread (sia il **main thread** che il thread secondario) deve leggere, modificare o scrivere su `myglobal`, deve prima **acquisire il mutex** (`pthread_mutex_lock`), eseguire le operazioni, e poi **rilasciare il mutex** (`pthread_mutex_unlock`).

Grazie a questa sincronizzazione, **solo un thread alla volta** può accedere alla variabile, garantendo che ogni incremento venga eseguito correttamente e che nessuna scrittura venga **persa**. Il risultato finale sarà quindi sempre **coerente**, come confermato dal possibile output:

```
myglobal è uguale a 40
```

Questo approccio dimostra che l'uso del **mutex elimina la race condition**, assicurando l'integrità dei dati condivisi anche in presenza di concorrenza.

Variabili di Condizione

Le **variabili di condizione** (*condition variables*) sono un meccanismo di **sincronizzazione** che consente ai **thread** di cooperare sospendendo temporaneamente la propria esecuzione in attesa del **verificarsi di una condizione** su dati condivisi. Vengono usate per evitare attese attive, cioè cicli di controllo continui sullo stato di una variabile.

Una variabile di condizione è **sempre associata a un mutex**, che garantisce **accesso esclusivo ai dati condivisi** durante il controllo e l'attesa della condizione.

Ogni variabile di condizione possiede una propria **coda di thread** e delle **funzioni** per segnalare la modifica della condizione e l'attesa. Una variabile di condizione è caratterizzata dalla variabile stessa, un **predicato** (la condizione o il valore che un thread controlla) e un **mutex** (permette di accedere al predicato in mutua esclusione). In C le variabili di condizione sono definite nella libreria `<pthread.h>` dal tipo di dato `pthread_cond_t`. Per creare una variabile di condizione **statica** prima si dichiara e poi si inizializza con il valore `PTHREAD_COND_INITIALIZER`.

L'inizializzazione **statica**, tramite la macro `PTHREAD_COND_INITIALIZER`, è adatta quando la variabile condizione è **globale** o **statica** e non richiede configurazioni particolari. Questa modalità consente una dichiarazione diretta e immediata, rendendola utile nei casi in cui la variabile non deve essere modificata dinamicamente e verrà utilizzata secondo il comportamento predefinito.

L'inizializzazione **dinamica**, ottenuta tramite la funzione `pthread_cond_init()`, è necessaria quando la variabile condizione è **locale**, oppure è stata **allocata dinamicamente** con `malloc`. Questo approccio offre maggiore **flessibilità** e consente l'uso di **attributi personalizzati** (tramite `pthread_condattr_t`) qualora sia richiesto un comportamento diverso da quello standard.

Quando non è più necessaria, va **distrutta** con `pthread_cond_destroy()` per evitare sprechi di memoria.

```
#include <pthread.h>
int pthread_cond_init (pthread_cond_t *cond, pthread_condattr_t
*attr);
int pthread_cond_destroy (pthread_cond_t *cond);
```

Queste due funzioni operano su una variabile di condizione **dinamica** e permettono rispettivamente di **inizializzarla** e **distruggerla**. Restituiscono **0** in caso di successo, altrimenti **-1** (flag `<errno.h>`). L'argomento `cond` è il puntatore alla variabile di condizione. L'argomento `attr` è la struttura degli attributi della variabile di condizione specificata (come per i mutex).

```
#include <pthread.h>
int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t
*mutex);
int pthread_cond_timedwait (pthread_cond_t *cond,
pthread_mutex_t *mutex, const struct timespec *abstime);
```

Restituiscono 0 in caso di successo, altrimenti un numero (flag <errno.h>).

La funzione `pthread_cond_wait()` mette il thread invocante in attesa fin quando la **condizione non si verifichi**. La funzione `pthread_cond_timedwait()` si comporta come la prima ma in aggiunta il thread viene **sbloccato** dopo un determinato **periodo di tempo**. L'argomento `cond` è la variabile di condizione inizializzata in precedenza. L'argomento `mutex` è il mutex che protegge il predicato e prima di essere passato alla funzione deve essere **inizializzato e bloccato**. Se il thread è messo in attesa della condizione, il mutex sarà **sbloccato** mentre quando ritorna dalla condizione sarà nuovamente **bloccato**. L'argomento `abstime` (secondi e nanosecondi) è il tempo di attesa del thread prima di sbloccarsi. Se la funzione ritorna perché è scaduto il tempo, verrà restituito l'errore `ETIMEDOUT`.

Le **funzioni di attesa** devono essere **inserite in un ciclo `while()`** con la condizione che non si deve verificare e prima di questo si deve **bloccare il mutex**. In questo modo si evitano comportamenti imprevisti dovuti ad eventuali **risvegli casuali** dei thread.

```
#include <pthread.h>
int pthread_cond_signal (pthread_cond_t *cond);
int pthread_cond_broadcast (pthread_cond_t *cond);
```

Queste due funzioni permettono rispettivamente di: **risvegliare un thread** presente nella coda di attesa (non si può decidere quale, dipende dalla politica di scheduling); **risvegliare tutti i thread** della coda di attesa. **Restituiscono 0** in caso di successo, altrimenti un numero (flag <errno.h>). L'argomento `cond` è la variabile di condizione.

Come si usa

1. **Creare il mutex** associato.
2. **Creare la variabile di condizione**: se è **statica** bisogna dichiarare una variabile locale `pthread_cond_t` e associarle il valore `PTHREAD_COND_INITIALIZER`. Se è **dinamica** bisogna dichiarare un puntatore `pthread_cond_t`, inizializzarne la memoria con `malloc()` e infine invocare `pthread_cond_init()`. Il secondo argomento è come la struttura degli attributi dei mutex e per default (se vale `NULL`) la variabile è **condivisa tra threads**.
3. Per **attendere** su una variabile di condizione bisogna: **bloccare il mutex** associato, dichiarare un **ciclo `while`** con condizione che non si deve verificare e **dentro** al ciclo invocare `pthread_cond_wait()` passandogli il mutex che è stato bloccato. **Fuori** il ciclo, se si è in sezione critica allora si eseguono le operazioni e alla fine si sblocca il mutex, altrimenti si sblocca subito il mutex e poi si eseguono le altre operazioni.
4. Negli altri thread, per **risvegliare un solo thread** in attesa sulla variabile si invoca `pthread_cond_signal()`, altrimenti per **risvegliare tutti** quelli in attesa si invoca `pthread_cond_broadcast()`.
5. Se la variabile di condizione è **dinamica**, bisogna invocare `pthread_cond_destroy()` per **eliminarla**. Se è **locale** non bisogna fare nulla, la deallocazione è gestita dal SO.

Tipica sequenza di azioni

Thread principale •Dichiara ed inizializza dati/variabili globali che richiedono sincronizzazione •Dichiara ed inizializza una variabile condizione •Dichiara ed inizializza un mutex associato •Crea thread A e B	
Thread A •Esegue fino al punto in cui una certa condizione deve verificarsi •Lock il mutex associato e controlla il valore di una variabile globale •Chiama pthread_cond_wait() per effettuare una wait bloccante (automaticamente e atomicamente corrisponde ad un unlock del mutex associato in modo tale che possa essere usato dal thread B) in attesa che il thread B lo risvegli, nel caso la condizione non sia verificata •Quando risvegliato, il lock del mutex avviene in modo automatico e atomico •Svolge le operazioni in mutua esclusione •Unlock il mutex in modo esplicito •Continua	Thread B Lavora Lock il mutex associato Modifica il valore della variabile globale su cui il thread A è in attesa Controlla il valore della variabile globale di attesa del thread A. Se si verifica la condizione desiderata, risveglia il thread A invocando pthread_cond_signal() Unlock il mutex Continua
Thread principale Join / Continua	

In uno scenario classico di cooperazione tra thread, il **thread principale** si occupa inizialmente di **dichiarare e inizializzare** tutte le **variabili globali** che richiedono sincronizzazione, comprese una **variabile di condizione** e un **mutex associato**. Successivamente, il thread principale **crea due thread** che coopereranno: il thread **A** e il thread **B**.

Il **thread A** esegue il suo codice fino a raggiungere un punto in cui è necessario **attendere che si verifichi una condizione**. A quel punto, **acquisisce il mutex** tramite `lock` e controlla lo stato di una variabile condivisa. Se la condizione desiderata non è ancora vera, chiama `pthread_cond_wait()`, sospendendosi in attesa. Durante questa operazione, il mutex viene **rilasciato in modo atomico** e il thread entra nella **coda di attesa** associata alla variabile condizione. Quando il **thread B** segnala che la condizione è avvenuta, il thread A si **risveglia** e il mutex viene automaticamente **riacquisito**. A questo punto, il thread A può eseguire le **operazioni protette in mutua esclusione**, e infine **rilascia il mutex esplicitamente** per proseguire la sua esecuzione.

Nel frattempo, il **thread B** lavora normalmente. Anche lui **acquisisce il mutex** per accedere alla variabile condivisa, modifica il suo valore e **verifica se la condizione attesa dal thread A è soddisfatta**. Se la condizione è vera, il thread B invoca `pthread_cond_signal()`, risvegliando il thread A. Dopo la segnalazione, il thread B **rilascia il mutex** e continua l'esecuzione.

Infine, una volta che entrambi i thread hanno concluso le loro attività, il **thread principale** esegue la **join** per attendere il completamento dei thread secondari, o prosegue con l'esecuzione del programma.

Esempio Mutex e variabili di condizione

Struttura dei dati e variabili globali

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```

// Struttura del messaggio
struct msg {
    struct msg *m_next; // Puntatore al prossimo messaggio (lista concatenata)
    /* altri dati utili nel messaggio possono stare qui */
};

// Coda di lavoro condivisa tra produttori e consumatori
struct msg *workq = NULL;

// Variabile di condizione per segnalare che ci sono messaggi pronti
pthread_cond_t qready = PTHREAD_COND_INITIALIZER;

// Mutex per proteggere l'accesso alla coda workq
pthread_mutex_t qlock = PTHREAD_MUTEX_INITIALIZER;

```

Thread **consumatore**: elabora messaggi dalla coda

```

void process_msg(void) {
    struct msg *mp;

    for (;;) {
        pthread_mutex_lock(&qlock); // Acquisisce il lock prima di accedere a workq

        // Attende finché la coda è vuota (predicato: workq != NULL)
        while (workq == NULL) {
            // Si sospende: viene inserito nella coda della variabile di condizione
            // Il mutex viene rilasciato automaticamente durante l'attesa
            pthread_cond_wait(&qready, &qlock);
        }

        // Qui è stato risvegliato: il mutex è già stato riacquisito
    }
}

```



```

// Preleva il primo messaggio dalla coda
mp = workq;
workq = mp->m_next;

pthread_mutex_unlock(&qlock); // Rilascia il mutex dopo l'accesso a workq

// Ora può elaborare il messaggio fuori dalla sezione critica
// (evita di bloccare altri thread inutilmente)
// ... elabora mp ...
}
}

```

Thread **produttore**: inserisce messaggi nella coda

```

void enqueue_msg(struct msg *mp) {
    pthread_mutex_lock(&qlock); // Protegge l'accesso alla coda

    mp->m_next = workq;      // Inserisce il nuovo messaggio in testa alla coda
    workq = mp;

    pthread_cond_signal(&qready); // Segnala che la coda non è più vuota (risveglia un thread in attesa)

    pthread_mutex_unlock(&qlock); // Rilascia il mutex
}

```

Considerazioni importanti

- La **condizione da attendere** è che `workq` **non sia NULL**. Questo è il **predicato**.
- Il ciclo `while` è essenziale: quando il thread si risveglia, potrebbe non essere l'unico in attesa (problema di **spurious wakeup** o risvegli anticipati).

- Il **mutex** protegge sia l'accesso al **predicato** che l'integrità della coda.
- La chiamata a `pthread_cond_signal()` **va fatta dopo aver aggiornato lo stato condiviso** (`workq`), così il thread risvegliato trova una coda valida.

Produttore Consumatore

Il problema **produttore-consumatore** descrive una situazione in cui uno o più **produttori** (thread o processi) generano dati che vengono successivamente elaborati da uno o più **consumatori**. I dati prodotti vengono trasmessi attraverso un meccanismo di **comunicazione tra processi (IPC)**. Quando tale comunicazione avviene tramite **memoria condivisa**, è fondamentale applicare una **sincronizzazione adeguata** per evitare condizioni di race o inconsistenze.

Nel caso specifico analizzato, si considerano **più thread produttori** e un **singolo thread consumatore** all'interno dello stesso processo. I dati condivisi sono memorizzati in un **array buff di interi**, che rappresenta l'area comune di lavoro tra produttori e consumatore.

Nel primo esempio, l'obiettivo è **sincronizzare esclusivamente i thread produttori**, evitando l'avvio del thread consumatore finché tutti i produttori non hanno terminato la generazione degli elementi.

Ogni produttore scrive sequenzialmente nell'array `buff`, impostando valori coerenti con la propria posizione: ad esempio, `buff[0]` sarà impostato a 0, `buff[1]` a 1, e così via.

Esempio 1

Il modello **produttore-consumatore** viene implementato all'interno di un singolo processo multithread, in cui **più thread produttori** generano dati da inserire in un **buffer condiviso**, mentre un **singolo thread consumatore** legge e verifica tali dati.

```
#include      ...

#define MAXNITEMS      1000000
#define MAXNTHREADS    100

int      nitems; //sola lettura per prod. e cons.
struct {
    pthread_mutex_t      mutex;
    int  buff[MAXNITEMS];
    int  nput;
    int  nval;
} shared = { PTHREAD_MUTEX_INITIALIZER };
void  *produce(void *), *consume(void *);
```

Le **variabili condivise** tra i thread sono contenute in una struttura `shared`, che include un **mutex**, un **array buff** dove vengono salvati gli elementi prodotti, e due contatori: `nput`, che rappresenta il prossimo indice libero nel buffer, e `nval`, il valore intero da scrivere. Il mutex è inizializzato staticamente con `PTHREAD_MUTEX_INITIALIZER`.

Main:

```
int main(int argc, char **argv)
{
    int                i, nthreads, count[MAXNTHREADS];

    pthread_t          tid_produce[MAXNTHREADS], tid_consume;
    if (argc != 3)
        {printf("usage: prodcons <#items> <#threads>");exit(-1);}
    nitems = MIN(atoi(argv[1]), MAXNITEMS);
    nthreads = MIN(atoi(argv[2]), MAXNTHREADS);

    /* inizia tutti i thread produttore */
    for (i = 0; i < nthreads; i++) {
        count[i] = 0;
        pthread_create(&tid_produce[i], NULL, produce, &count[i]);
    }

    /* aspetta tutti i thread produttore */
    for (i = 0; i < nthreads; i++) {
        pthread_join(tid_produce[i], NULL);
        printf("count[%d] = %d\n", i, count[i]);
    }

    /* inizia e poi aspetta il thread consumatore */
    pthread_create(&tid_consume, NULL, consume, NULL);
    pthread_join(tid_consume, NULL);
    exit(0);}
```

Nel main, il programma riceve da riga di comando il numero di elementi da produrre e il numero di thread da utilizzare. Viene quindi creato un array di contatori `count[]`, uno per ciascun thread produttore, inizializzato a zero. Ogni produttore riceve in ingresso un puntatore al proprio elemento dell'array `count`, che incrementa ogni volta che scrive un elemento nel buffer.

Produttore

```

void *produce(void *arg)
{
    for ( ; ; ) {
        pthread_mutex_lock(&shared.mutex);
        if (shared.nput >= nitems) {
            pthread_mutex_unlock(&shared.mutex);
            return(NULL); /* array pieno */
        }
        shared.buff[shared.nput] = shared.nval;
        shared.nput++;
        shared.nval++;
        pthread_mutex_unlock(&shared.mutex);
        *((int *) arg) += 1;
    }
}

```

Ogni thread produttore entra in un ciclo infinito in cui tenta di scrivere nel buffer. Prima di accedere alla struttura condivisa, **acquisisce il mutex**, poi verifica se la coda è piena confrontando `shared.nput` con il numero massimo di elementi. Se il buffer è pieno, rilascia il mutex e termina. Altrimenti, scrive il valore `nval` nella posizione `nput` dell'array, aggiorna entrambi i contatori (`nput` e `nval`) e poi **rilascia il mutex**. Infine, incrementa il proprio contatore privato.

La regione critica per i produttori consiste nel verificare se il buffer è pieno. Proteggiamo questa porzione di codice con il mutex, assicurandoci di sbloccarlo appena finito il controllo e l'esecuzione delle relative istruzioni.

La **regione critica** è limitata all'accesso al buffer e ai contatori condivisi. Le operazioni sui contatori locali `count[]`, passati tramite argomento, non necessitano protezione perché ogni thread scrive su una propria posizione.

Consumatore

```

void *
consume(void *arg)
{
    int      i;

    for (i = 0; i < nitems; i++) {
        if (shared.buff[i] != i)
            printf("buff[%d] = %d\n", i, shared.buff[i]);
    }
    return(NULL);
}

```

Una volta terminata l'esecuzione di tutti i thread produttori (gestita tramite `pthread_join`), viene avviato il **thread consumatore**, che scorre l'intero buffer e **verifica la correttezza** dei dati confrontando ciascun valore con il proprio indice (`buff[i] == i`). In caso di discrepanza, stampa un messaggio d'errore.

Locking contro Waiting

L'obiettivo è evidenziare che i **mutex** sono strumenti adeguati per garantire **esclusione reciproca (locking)**, ma non sono sufficienti per gestire situazioni di **attesa sincronizzata (waiting)**. Per dimostrarlo, si modifica l'esempio del modello **produttore-consumatore** facendo partire il **thread consumatore subito dopo l'avvio dei produttori**.

Questa modifica permette al consumatore di iniziare a **elaborare i dati appena vengono prodotti**, senza attendere la conclusione di tutti i produttori. Tuttavia, per evitare che il consumatore legga **dati non ancora disponibili**, è necessario **sincronizzare la sua esecuzione** con quella dei produttori. Solo così si può garantire che il consumatore **acceda esclusivamente a dati già scritti nel buffer**.

Questo scenario dimostra che, mentre il **mutex** serve a proteggere l'accesso ai dati durante la scrittura o la lettura, per implementare un **meccanismo di attesa attiva o sospesa**, è necessario introdurre **variabili di condizione** o **semafori**, strumenti più adatti alla gestione del **waiting**.

Esempio 2 Produttore-Consumatore

Nel secondo esempio produttore-consumatore si evidenzia un limite importante dell'uso esclusivo dei **mutex**. I mutex sono strumenti adatti al **locking**, cioè alla **protezione dell'accesso concorrente** a dati condivisi, ma non sono sufficienti per gestire correttamente l'**attesa attiva**. Questo problema emerge quando il **thread consumatore** viene avviato subito dopo i produttori, con l'intento di iniziare ad elaborare gli elementi non appena disponibili, invece di attendere che tutti i produttori abbiano terminato.

Main

```
int main(int argc, char **argv)
{
    int i, nthreads, count[MAXNTHREADS];
```

```

pthread_t      tid_produce[MAXNTHREADS], tid_consume;
if (argc != 3)
    {printf("usage: prodcons2 <#items> <#threads>");exit(-1);}
nitems = MIN(atoi(argv[1]), MAXNITEMS);
nthreads = MIN(atoi(argv[2]), MAXNTHREADS);

for (i = 0; i < nthreads; i++) {
    count[i] = 0;
    pthread_create(&tid_produce[i], NULL, produce, &count[i]);
}
pthread_create(&tid_consume, NULL, consume, NULL);

/* aspetta tutti i produttori e il consumatore */
for (i = 0; i < nthreads; i++) {
    pthread_join(tid_produce[i], NULL);
    printf("count[%d] = %d\n", i, count[i]);
}
pthread_join(tid_consume, NULL);

exit(0);

```

Produce (resta uguale)

```

void *produce(void *arg)
{
    for ( ; ; ) {
        pthread_mutex_lock(&shared.mutex);
        if (shared.nput >= nitems) {
            pthread_mutex_unlock(&shared.mutex);
            return(NULL); /* array pieno */
        }
        shared.buff[shared.nput] = shared.nval;
        shared.nput++;
        shared.nval++;
        pthread_mutex_unlock(&shared.mutex);
        *((int *) arg) += 1;
    }
}

```

Consume

```

void *consume(void *arg)
{
    int      i;

    for (i = 0; i < nitems; i++) {
        consume_wait(i);
        if (shared.buff[i] != i)
            printf("buff[%d] = %d\n", i, shared.buff[i]);
    }
    return(NULL);
}

```

Consume_wait

```
void consume_wait(int i)
{
    for ( ; ; ) {

        pthread_mutex_lock(&shared.mutex);
        if (i < shared.nput) {
            pthread_mutex_unlock(&shared.mutex);
            return;          /* un elemento è pronto */
        }
        pthread_mutex_unlock(&shared.mutex);
    }
}
```

Per evitare che il consumatore acceda a **dati non ancora generati**, viene introdotta la funzione `consume_wait`, che **verifica** se l'*i*-esimo elemento del buffer è stato prodotto. Per farlo, confronta il valore *i* con `nput`, che rappresenta l'indice del prossimo elemento da scrivere. Tuttavia, essendo `nput` una **variabile condivisa**, la lettura deve avvenire **sotto protezione del mutex**, per evitare letture inconsistenti durante l'aggiornamento da parte dei produttori.

Il problema principale è che, nel momento in cui l'elemento non è disponibile, `consume_wait` entra in un **ciclo continuo** di controllo (polling), in cui il mutex viene acquisito e rilasciato ripetutamente fino a quando la condizione `i < nput` risulta vera. Questo approccio è inefficiente: comporta un **alto spreco di tempo di CPU**, poiché il consumatore continua ad attivarsi inutilmente.

Per gestire questa situazione in modo corretto è necessario un **meccanismo di attesa sospesa**, ovvero una **variabile di condizione**. Mentre i **mutex** servono a garantire l'esclusione mutua, le **variabili di condizione** sono progettate per **bloccare temporaneamente l'esecuzione** di un thread fino al verificarsi di un evento specifico. A ciascuna variabile condizione è sempre associato un mutex, ed è tramite la funzione `pthread_cond_wait()` che si effettua l'attesa, in combinazione con `pthread_cond_signal()` o `pthread_cond_broadcast()` per risvegliare i thread in attesa.

In sintesi, questo esempio dimostra che **locking e waiting** sono **due forme diverse di sincronizzazione** e richiedono strumenti distinti. Per bloccare l'accesso ai dati condivisi è sufficiente il mutex, ma per sospendere in modo efficiente l'esecuzione fino al verificarsi di una condizione, servono le **condition variables**.

Esempio 3 produttore-consumatore

```
#include ...
#define MAXNITEMS 1000000
#define MAXNTHREADS 100
```

```

        /* globali condivise dai thread */

int            nitems;        /* sola lettura per prod. e cons. */
int            buff[MAXNITEMS];

struct {
    pthread_mutex_t    mutex;

    int            nput;    // indice successivo in cui memorizzare

    int            nval;    // valore successivo da memorizzare
} put = { PTHREAD_MUTEX_INITIALIZER };

struct {
    pthread_mutex_t    mutex;

    pthread_cond_t    cond;

    int            nready; // numero a disposizione del cons.
} nready = { PTHREAD_MUTEX_INITIALIZER, PTHREAD_COND_INITIALIZER };

/* fine globali */

void            *produce(void *), *consume(void *);

```

Nel terzo esempio, viene migliorata l'implementazione del problema produttore-consumatore, aggiungendo una vera gestione dell'**attesa passiva** tramite **variabili di condizione**. Le strutture dati vengono suddivise per chiarezza e controllo: la struttura **put** contiene le variabili `nput` e `nval`, usate dai **produttori**, ed è protetta da un mutex; la struttura **nready** è condivisa tra **produttori e consumatore** e contiene un contatore `nready`, una variabile di condizione `cond` e il mutex associato.

Main

```

int
main(int argc, char **argv)
{

    int            i, nthreads, count[MAXNTHREADS];
    pthread_t      tid_produce[MAXNTHREADS], tid_consume;

    if (argc != 3)
        {printf("usage: prodcons3 <#items> <#threads>");exit(-1);}

    nitems = MIN(atoi(argv[1]), MAXNITEMS);
    nthreads = MIN(atoi(argv[2]), MAXNTHREADS);

        /* crea tutti i produttori ed un consumatore */

    for (i = 0; i < nthreads; i++) {
        count[i] = 0;

        pthread_create(&tid_produce[i], NULL, produce, &count[i]);
    }

    pthread_create(&tid_consume, NULL, consume, NULL);

        /* aspetta tutti i produttori ed il consumatore */

    for (i = 0; i < nthreads; i++) {
        pthread_join(tid_produce[i], NULL);
        printf("count[%d] = %d\n", i, count[i]);
    }

    pthread_join(tid_consume, NULL);

    exit(0);}

```


Produttore

```
void *produce(void *arg)
{
    for ( ; ; ) {

        pthread_mutex_lock(&put.mutex);
        if (put.nput >= nitems) {
            pthread_mutex_unlock(&put.mutex);
            return(NULL);          /* array pieno */
        }
        buff[put.nput] = put.nval;
        put.nput++;
        put.nval++;
        pthread_mutex_unlock(&put.mutex);
        pthread_mutex_lock(&nready.mutex);
        if (nready.nready == 0)
            pthread_cond_signal(&nready.cond);
        nready.nready++;
        pthread_mutex_unlock(&nready.mutex);
        *((int *) arg) += 1;
    }
}
```

Durante l'esecuzione del **thread produttore**, si accede in mutua esclusione alla struttura `put` per scrivere un nuovo elemento nell'array `buff`, incrementando poi `nput` e `nval`. Dopo questa operazione, il thread accede alla struttura `nready` e incrementa `nready.nready`, che rappresenta il numero di elementi pronti per la lettura. Se prima dell'incremento `nready.nready` era uguale a zero, viene invocata la funzione `pthread_cond_signal()` per **risvegliare il consumatore** in attesa.

Consumatore

```
void *consume(void *arg)
{
    int          i;

    for (i = 0; i < nitems; i++) {
        pthread_mutex_lock(&nready.mutex);
        while (nready.nready == 0)
            pthread_cond_wait(&nready.cond, &nready.mutex);
        nready.nready--;
        pthread_mutex_unlock(&nready.mutex);

        if (buff[i] != i)
            printf("buff[%d] = %d\n", i, buff[i]);
    }
    return(NULL);
}
```

Nel **thread consumatore**, prima di accedere al buffer, si controlla che `nready.nready` sia maggiore di zero. Se il valore è zero, viene chiamata la funzione `pthread_cond_wait()`, che **blocca il thread consumatore** e rilascia automaticamente il mutex associato. Quando un produttore produce un nuovo elemento e segnala la condizione, il consumatore si risveglia, decrementa il contatore `nready.nready`, e continua l'esecuzione.

Questo meccanismo consente una **sincronizzazione efficiente**: il consumatore non effettua più polling attivo ma entra in **sleep** finché non riceve una notifica dai produttori. La variabile `nready` funge da **contatore di elementi disponibili** ed è la condizione monitorata. Il segnale viene generato solo quando il contatore cambia da 0 a 1, ottimizzando le notifiche.

Esempio (Uso dei Mutex)

Questo esempio mostra l'utilizzo di un **mutex** per proteggere una **variabile condivisa** tra due thread. La variabile `DATA` viene incrementata da entrambi i thread (`thread1_process` e `thread2_process`), e l'accesso a questa variabile avviene all'interno di una **sezione critica** protetta dal **mutex m**.

thread1_process e thread2_process

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```

#define MAX 10

pthread_mutex_t M; /* def.mutex condiviso tra thread */

int DATA=0;      /* variabile condivisa */

int accessi1=0; // num. accessi thread 1 alla sez. critica
int accessi2=0; // num. accessi thread 2 alla sez. critica

void *thread1_process (void * arg) {

    int k=1;

    while(k) {

        accessi1++;

        pthread_mutex_lock(&M);

        DATA++;

        k=(DATA>=MAX?0:1);

        pthread_mutex_unlock(&M);

        printf("accessi di T1: %d\n", accessi1);

        sleep(1);

    }

    pthread_exit (0);

}

```

Ogni thread entra in un ciclo in cui incrementa un contatore di accessi (`accessi1` o `accessi2`), acquisisce il **lock** sul mutex, modifica la variabile `DATA`, poi verifica se `DATA` ha raggiunto un valore massimo (`MAX=10`). In tal caso, imposta una variabile di uscita (`k`) a 0 per terminare il ciclo. Successivamente, rilascia il **mutex** e stampa il numero di accessi.

Main

```

int main () {

    pthread_t th1, th2; // il mutex è inizialmente libero

```

```

pthread_mutex_init (&M, NULL);
if (pthread_create(&th1, NULL, thread1_process, NULL) < 0) {
    fprintf (stderr, "errore creazione per thread 1\n");
    exit (1);
}
if (pthread_create(&th2, NULL,thread2_process,NULL) < 0)
{
    fprintf (stderr, "errore creazione per thread 2\n");
    exit (1);
}
pthread_join (th1, NULL);
pthread_join (th2, NULL);
printf("Accessi: T1: %d, T2 %d\n",accessi1,accessi2);
printf("Totale accessi: %d\n",DATA);
exit(0);
}

```

Nel main, i thread vengono creati e avviati con `pthread_create()`, dopo che il **mutex è inizializzato** tramite `pthread_mutex_init()`. Alla fine, il programma aspetta la terminazione dei due thread con `pthread_join()` e stampa il numero totale di accessi e il valore finale di `DATA`.

L'uso del **mutex** garantisce che l'incremento della variabile `DATA` avvenga in modo **mutuamente esclusivo**, evitando condizioni di race. Grazie al meccanismo di lock/unlock, si garantisce la **correttezza e coerenza** dell'accesso alla variabile condivisa.

Capitolo 18 – Semafori Posix

Introduzione

I **semafori** sono primitive di sincronizzazione utilizzate per coordinare l'accesso a risorse condivise tra processi o thread. Si parte dal concetto di **semaforo binario**, che può assumere solo i valori **0 (bloccato)** e **1 (sbloccato)**.

Esistono tre principali tipologie di semafori: i **semafori Posix**, non gestiti direttamente dal kernel, che si suddividono in due categorie (quelli basati su nome e quelli in memoria condivisa), e i **semafori System V**, che sono mantenuti all'interno del kernel.

Operazioni sui semafori

Un processo può eseguire tre operazioni fondamentali sui semafori. La **creazione** include anche l'inizializzazione del valore di partenza.

```
while (semaphore_value == 0)
; /* wait, ovvero blocca il processo */
semaphore_value--;
```

L'operazione di **wait** (o **P**) blocca il processo se il valore del semaforo è zero, altrimenti lo decrementa, ed è fondamentale che questa operazione sia **atomica**. L'operazione di **post** (o **V**, o **signal**) incrementa il valore del semaforo e risveglia eventuali processi in attesa; anche in questo caso l'operazione deve essere **atomica**.

Semaforo contatore

Il **semaforo contatore** è una generalizzazione del semaforo binario, con un valore che varia tra 0 e un limite superiore (almeno 32767 per Posix). Serve a contare le istanze disponibili di una risorsa, e quindi rappresenta il numero di risorse disponibili. L'operazione di **wait** attende che il valore sia maggiore di zero per poterlo decrementare, mentre l'operazione di **post** lo incrementa e può risvegliare i processi in attesa.

Semaforo Binario e mutua esclusione

<code>inizializza il mutex;</code>	<code>inizializza il semaforo a 1;</code>
<code>pthread_mutex_lock(&mutex);</code>	<code>sem_wait(&sem);</code>
<code> regione critica;</code>	<code> regione critica;</code>
<code>pthread_mutex_unlock(&mutex);</code>	<code>sem_post(&sem);</code>

Un **semaforo binario** può essere utilizzato per ottenere la **mutua esclusione**, proprio come un **mutex**. Si inizializza a 1, e la chiamata a **sem_wait()** attende che il valore sia maggiore di 0 per poi decrementarlo, entrando nella **regione critica**. Al termine, si usa **sem_post()** per incrementare nuovamente il valore e permettere l'accesso ad altri thread o processi.

Una **differenza importante** tra semaforo e mutex è che, mentre il mutex deve essere **rilasciato dallo stesso thread** che lo ha acquisito, il semaforo può essere **modificato da thread diversi**: chi esegue `sem_post()` non deve essere lo stesso che ha fatto `sem_wait()`. Questa proprietà rende i semafori adatti anche per la **sincronizzazione tra thread distinti**, come nel classico problema del **produttore-consumatore**.

Esempio Produttore – Consumatore

Produttore

```
inizializza semaforo get a 0;
inizializza semaforo put a 1;

for( ; ; ) {
    sem_wait(&put);
    inserimento dati nel buffer;
    sem_post(&get);
}
```

Consumatore

```
for( ; ; ) {
    sem_wait(&get);
    elabora dati dal buffer;
    sem_post(&put);
}
```

Nel **caso semplificato** del produttore-consumatore con **un solo elemento nel buffer condiviso**, si usano due semafori binari: **put** (inizializzato a 1) e **get** (inizializzato a 0). Il **produttore** attende su `sem_wait(&put)`, produce un dato e poi esegue `sem_post(&get)`. Il **consumatore** attende su `sem_wait(&get)`, consuma il dato e chiama `sem_post(&put)`.

Il ciclo di esecuzione si sviluppa così:

1. Il **consumatore si blocca** inizialmente perché il semaforo `get` è a 0.
2. Il **produttore** entra, attende su `put`, lo decrementa e produce un dato.
3. Esegue `sem_post(&get)`, risvegliando il **consumatore** che può ora consumare il dato.
4. Il consumatore chiama `sem_post(&put)` e il ciclo riparte.

L'ipotesi è che, anche se un thread è marcato come **pronto all'esecuzione** a seguito di un `sem_post()`, il thread chiamante continui comunque la propria esecuzione. Questo non altera la **correttezza della sincronizzazione**, perché l'ordine di esecuzione tra thread pronti è irrilevante ai fini logici del meccanismo.

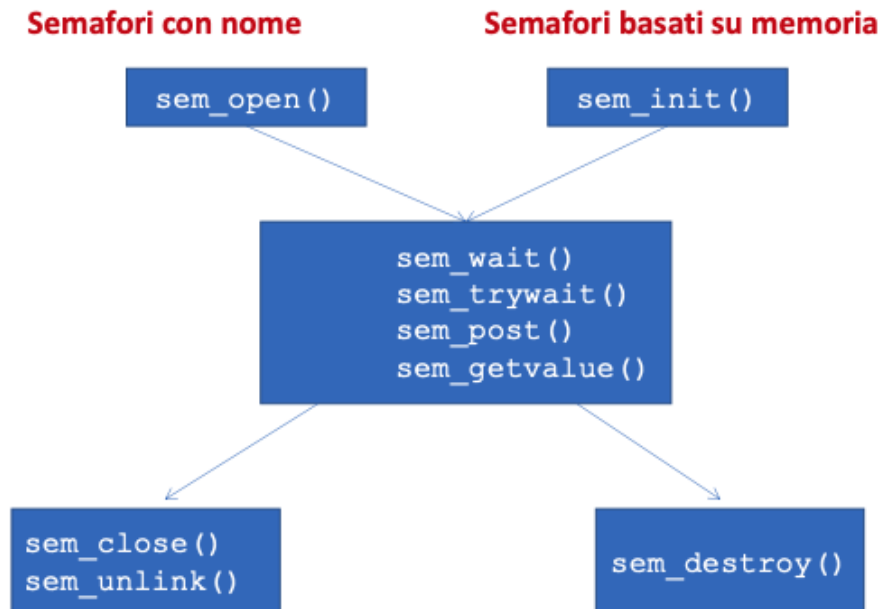
Differenze tra meccanismi di sincronizzazione

Esistono **differenze importanti** tra **semafori**, **mutex** e **variabili condizione**. Un **mutex** deve sempre essere **rilasciato dallo stesso thread** che lo ha acquisito, mentre un **semaforo** può essere **modificato da un thread diverso** da quello che ha eseguito la `wait()`. Inoltre, il mutex ha **solo due stati** (bloccato o sbloccato), come un **semaforo binario**, mentre un semaforo può anche essere un **contatore**.

Un altro punto cruciale è che, poiché un **semaforo** mantiene **uno stato interno (il valore)**, un'operazione `post()` viene **sempre memorizzata** anche se nessun thread è in attesa. Al contrario, un segnale su una **variabile condizione** viene **perso** se non ci sono thread in attesa nel momento in cui viene emesso.

Funzioni per i semafori Posix

Nel contesto dei **semafori POSIX**, esistono due tipologie: quelli **con nome**, usati per la comunicazione tra processi attraverso un identificatore simbolico (`sem_open`, `sem_close`, `sem_unlink`) e quelli **basati su memoria condivisa**, che richiedono l'inizializzazione con `sem_init` e la distruzione con `sem_destroy`.



Le funzioni comuni a entrambe le tipologie sono:

- `sem_wait()` e `sem_trywait()` per bloccare o tentare di bloccare il semaforo.
- `sem_post()` per incrementarne il valore.
- `sem_getvalue()` per ottenere il valore corrente.

Semafori Posix Basati su Nome

I **semafori POSIX con nome** permettono la **sincronizzazione tra processi** diversi (oltre che tra thread) tramite un **nome simbolico** riconosciuto dal sistema, e si creano o aprono tramite la funzione `sem_open()`.

Creazione di un semaforo

```
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, mode_t mode, int value);

/* Restituisce un puntatore al semaforo se OK, SEM_FAILED in caso di errore */
```

Il primo parametro di `sem_open()` è il **nome IPC Posix**, che rappresenta un percorso (reale o simbolico). Il secondo parametro, `oflag`, può essere:

- 0 per aprire un semaforo esistente,
- `O_CREAT` per crearlo se non esiste,
- `O_CREAT | O_EXCL` per forzare un errore se esiste già.

Se si usa `O_CREAT`, si devono specificare anche i **permessi (mode)** e il **valore iniziale** del semaforo. Quest'ultimo non può superare **`SEM_VALUE_MAX`** (almeno 32767); in genere, i **semafori binari** iniziano da 1, mentre i **semafori contatore** possono avere un valore maggiore.

L'uso combinato di `O_CREAT` e `O_EXCL` garantisce che la creazione avvenga solo se il semaforo **non esiste già**, altrimenti si genera un errore. Se si usa solo `O_CREAT`, un semaforo già esistente viene semplicemente aperto senza errore.

La funzione **ritorna un puntatore** di tipo `sem_t *`, che poi può essere utilizzato con le altre funzioni POSIX:

`sem_wait()`, `sem_trywait()`, `sem_post()`, `sem_getvalue()`, `sem_close()` e `sem_unlink()`.

Chiusura di un semaforo

```
#include <semaphore.h>

int sem_close(sem_t *sem);

/* Restituisce 0 se OK, -1 in caso di errore */
```

Un semaforo creato o aperto con `sem_open()` viene chiuso usando la funzione `sem_close()`, che libera le risorse associate **a livello di processo**.

La chiusura di un semaforo con `sem_close()` può avvenire **esplicitamente** o **automaticamente** alla terminazione del processo che aveva aperto il semaforo, indipendentemente dal fatto che sia terminato normalmente o per errore.

È importante notare che `sem_close()` **non elimina il semaforo dal sistema**: il nome associato al semaforo **rimane persistente a livello di kernel**, così come il valore interno del semaforo, anche se nessun processo lo ha attualmente aperto.

Per rimuovere completamente un semaforo con nome, è necessario usare `sem_unlink()`, che dissocia il nome e dealloca il semaforo solo quando non è più referenziato da alcun processo.

Persistenza degli oggetti di IPC

La **persistenza degli oggetti di IPC** (Inter-Process Communication) indica per quanto tempo un oggetto esiste nel sistema. Si possono distinguere tre livelli:

- **Persistenza di processo**: l'oggetto esiste finché l'ultimo processo che lo ha aperto lo chiude (es. pipe, FIFO).

- **Persistenza a livello kernel:** l'oggetto rimane nel sistema finché non viene cancellato esplicitamente o fino al riavvio (es. **semafori System V** e **semafori Posix**). È il livello minimo di persistenza richiesto per gli oggetti POSIX.
- **Persistenza a livello di file system:** l'oggetto persiste anche oltre il riavvio del sistema finché non viene cancellato esplicitamente. Alcune implementazioni POSIX permettono ciò per semafori e memoria condivisa.

Rimozione di un semaforo

```
#include <semaphore.h>

int sem_unlink(const char *name);

/* Restituisce 0 se OK, -1 in caso di errore */
```

Per quanto riguarda la **rimozione di un semaforo POSIX con nome**, essa si effettua con la funzione `sem_unlink()`, che rimuove il **nome** dal file system. Tuttavia, la rimozione effettiva dell'oggetto avviene solo **dopo che tutti i riferimenti attivi al semaforo sono chiusi** con `sem_close()`.

Ogni semaforo mantiene un **contatore di riferimento** che tiene traccia delle aperture ancora attive. Se il nome viene rimosso quando il contatore è maggiore di zero, il semaforo continua a esistere finché tutti i processi non lo chiudono.

sem_wait e sem_trywait

```
#include <semaphore.h>

int sem_wait(sem_t *sem);

int sem_trywait(sem_t *sem);

/* Restituisce 0 se OK, -1 in caso di errore */
```

La funzione `sem_wait()` controlla il valore del semaforo: se è maggiore di 0, lo **decrementa** e prosegue l'esecuzione; se è 0, il **thread viene bloccato** finché il valore non diventa positivo. L'operazione è **atomica** per garantire la corretta sincronizzazione.

La funzione `sem_trywait()` ha lo stesso comportamento di `sem_wait()`, ma **non blocca** il thread: se il valore del semaforo è 0, restituisce l'errore **EAGAIN**.

`sem_wait()` può anche terminare prematuramente se il thread viene **interrotto da un segnale**, in tal caso ritorna con errore **EINTR**.

sem_post e sem_getvalue

```
#include <semaphore.h>

int sem_post(sem_t *sem);

int sem_getvalue(sem_t *sem, int *valp);

/* Restituisce 0 se OK, -1 in caso di errore */
```

La funzione `sem_post()` **incrementa il valore del semaforo** di 1. Se ci sono thread bloccati in attesa, uno di essi viene risvegliato e marcato come pronto all'esecuzione.

La funzione `sem_getvalue()` consente di leggere il **valore corrente** del semaforo nel puntatore `*valp` passato come parametro. Se il semaforo è bloccato, lo standard **Posix.1-2001** prevede due comportamenti:

- **Restituire 0**, come fanno Linux e OpenSolaris.
- Oppure un **numero negativo**, il cui valore assoluto rappresenta il **numero di thread bloccati** in attesa che il semaforo sia sbloccato.

Esempio

Il programma `semcreate` dimostra l'uso dei **semafori POSIX con nome** e la loro **persistenza a livello kernel**, che consente l'interazione tra **programmi multipli**. Questo programma consente la **creazione di un semaforo** con due opzioni:

- `-e` attiva la creazione esclusiva (`O_EXCL`), fallendo se il semaforo esiste già.
- `-i` permette di specificare un **valore iniziale** per il semaforo, altrimenti il valore di default è 1.

semcreate

```
#define FILE_MODE S_IRUSR|S_IWUSR ...

int main(int argc, char **argv)
{ int    c, flags;

  sem_t  *sem;
  unsigned int  value;
  flags = O_CREAT;
  value = 1;
  while ( (c = getopt(argc, argv, "ei:")) != -1) {
    switch (c) {
      case 'e':
        flags |= O_EXCL;
        break;
      case 'i':
        value = atoi(optarg);
        break;
    }
  }
  if (optind != argc-1){
    fprintf(stderr, "usage: semcreate [-e] [-i initialValue] <name>");
    exit(-1);}
  sem = sem_open(argv[optind], flags, FILE_MODE, value); sem_close(sem); exit(0);}
```

Funzione getopt

```
#include <unistd.h>
```

```
int getopt(int argc, char *const argv[], const char *optstring);  
extern char *optarg;  
extern int optind;
```

La funzione `getopt()`, definita in `<unistd.h>`, permette di **analizzare in modo strutturato gli argomenti da riga di comando**. Viene usata per identificare e gestire **opzioni** specificate nel programma (es. `-e`, `-i`, ecc.).

La funzione prende come parametri:

- Il numero di argomenti (`argc`)
- L'array degli argomenti (`argv[]`)
- Una stringa di caratteri (`optstring`) che definisce le opzioni valide. Se un carattere è seguito da : (due punti), significa che l'opzione **richiede un argomento** (es. `-i 3`).

La funzione restituisce il carattere dell'opzione trovata oppure `-1` se non ci sono più opzioni. L'argomento associato (se presente) è disponibile nella variabile globale `optarg`. L'indice del prossimo elemento da elaborare in `argv[]` è memorizzato in `optind`, inizializzato automaticamente a 1.

`getopt()` è quindi uno **strumento essenziale per la gestione elegante degli input utente**, soprattutto nei programmi che richiedono flessibilità e parametri configurabili.

La funzione `sem_open()` è chiamata con **quattro parametri**:

1. Nome del semaforo
2. Flag (`O_CREAT` e eventualmente `O_EXCL`)
3. Modalità di permessi (`FILE_MODE`)
4. Valore iniziale (usato solo se il semaforo non esiste già)

Infine, il semaforo viene **chiuso con `sem_close()`**, ma non viene rimosso dal sistema: rimane presente nel file system finché non viene eliminato con `sem_unlink()`.

Semunlink

```
#include ...  
  
int main(int argc, char **argv)  
{  
    if (argc != 2){  
        fprintf(stderr, "usage: semunlink <name>");  
        exit(-1);  
    }  
  
    sem_unlink(argv[1]);  
  
    exit(0);  
}
```

Il programma `semunlink` serve per **rimuovere un semaforo POSIX con nome** dal sistema. Controlla che l'utente abbia fornito un solo argomento (il nome del semaforo), e poi chiama `sem_unlink()` per rimuoverlo. L'operazione libera la risorsa a livello di file system, ma il semaforo rimane attivo finché tutti i riferimenti (processi) non lo chiudono con `sem_close()`.

Semgetvalue

```
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>

int
main(int argc, char **argv)
{
    sem_t *sem;
    int val;

    if (argc != 2) {
        fprintf(stderr, "usage: semgetvalue <name>");
        exit(-1);
    }
    sem = sem_open(argv[1], 0);
    sem_getvalue(sem, &val);
    printf("value = %d\n", val);
    exit(0);
}
```

Il programma `semgetvalue` apre un semaforo **già esistente** (usando `sem_open()` con flag 0, senza `O_CREAT`) e **ne legge il valore corrente** con `sem_getvalue()`, stampandolo a video. Questo è utile per **verificare lo stato attuale** del semaforo, ad esempio in fase di debug o monitoraggio.

In entrambi i casi, se il numero di argomenti forniti non è corretto (`argc != 2`), viene mostrato un messaggio d'uso e il programma termina con errore.

Semwait

```
#include ...

int
main(int argc, char **argv)
{
    sem_t *sem;
    int    val;

    if (argc != 2){
        fprintf(stderr, "usage: semwait <name>");
        exit(-1);}
    sem = sem_open(argv[1], 0);
    sem_wait(sem);
    sem_getvalue(sem, &val);
    printf("pid %ld has semaphore, value = %d\n", (long) getpid(), val);

    pause(); /* bloccato fino a terminazione da utente */
    exit(0);
}
```

Il programma `semwait` apre un semaforo esistente tramite `sem_open()` (senza `O_CREAT`), quindi esegue `sem_wait()`, che blocca il thread chiamante **se il valore del semaforo è 0**. Quando il semaforo diventa disponibile, lo **decrementa** e il programma prosegue. Subito dopo, legge il valore attuale del semaforo con `sem_getvalue()` e lo stampa a video insieme al **PID del processo**. Infine, chiama `pause()`, che sospende l'esecuzione **fino a un segnale**, bloccando il processo.

Sempost

```
#include ...

int
main(int argc, char **argv)
{
    sem_t *sem;
    int    val;

    if (argc != 2){
        fprintf(stderr, "usage: sempost <name>");
        exit(-1);
    }
    sem = sem_open(argv[1], 0);
    sem_post(sem);
    sem_getvalue(sem, &val);
    printf("value = %d\n", val);

    exit(0);
}
```

Il programma `sempost`, invece, apre un semaforo con `sem_open()`, esegue `sem_post()` (incrementandone il valore di 1) e stampa il valore aggiornato del semaforo. Serve tipicamente per **risvegliare un processo bloccato** da `sem_wait()`.

Questi due strumenti illustrano chiaramente l'interazione tra semafori:

- `semwait` simula un processo che **richiede** una risorsa e attende;
- `sempost` simula un processo che **rilascia** una risorsa e sblocca chi attendeva.

Problema Produttore – Consumatore

Il **problema produttore-consumatore** riguarda la sincronizzazione tra **più thread produttori**, che inseriscono elementi in un **buffer condiviso**, e un **thread consumatore**, che preleva tali elementi. In una prima versione semplificata, il consumatore iniziava solo dopo che tutti i produttori avevano finito, ed era sufficiente un **mutex** per garantire la sincronizzazione. In una versione successiva, il consumatore poteva iniziare anche mentre i produttori erano attivi, richiedendo l'uso combinato di **mutex** e **variabili di condizione**.

Estendendo il problema, si utilizza un **buffer circolare**: quando il produttore raggiunge l'ultima posizione, riparte dalla prima. Anche il consumatore segue lo stesso schema. Questo impone nuove regole di sincronizzazione: il **produttore non deve superare il consumatore**, cioè non può scrivere in posizioni non ancora lette.

Tre **condizioni fondamentali** devono essere rispettate: il **consumatore non può leggere se il buffer è vuoto**, il **produttore non può scrivere se il buffer è pieno**, e l'**accesso alle variabili condivise** (come indici e contatori) deve essere **protetto** per evitare condizioni di competizione (**race condition**).

La soluzione proposta utilizza **tre semafori**: un **semaforo binario** chiamato **mutex**, inizializzato a 1, che protegge l'accesso al buffer durante lettura e scrittura; un **semaforo contatore** chiamato **nempty**, inizializzato al numero di posizioni disponibili (**NBUFF**), che controlla quanti slot vuoti ci sono; e un **semaforo contatore** chiamato **nstored**, inizializzato a 0, che tiene traccia di quante posizioni del buffer contengono dati pronti per essere letti.

Nel caso specifico illustrato, il produttore memorizza nel buffer interi da **0 a nitems**, mentre il consumatore li legge e verifica che siano corretti. Il **buffer è circolare** e i controlli avvengono tramite stampa su standard output.

Variabili globali

```
#include      ...

#define NBUFF      10
#define SEM_MUTEX  "/mutex"
#define SEM_NEMPTY "/nempty"
#define SEM_NSTORED "/nstored"

int    nitems; /* sola lettura per prod. e cons. */
struct {      /* dati condivisi da prod. e cons. */
    int  buff[NBUFF];
    sem_t *mutex, *nempty, *nstored;
} shared;

void    *produce(void *), *consume(void *);
```

Nel problema produttore-consumatore, le **variabili condivise** tra i thread sono raccolte in una struttura globale chiamata `shared`. Questa contiene il **buffer circolare** di dimensione `NBUFF`, definito a 10, e i **puntatori ai tre semafori**: `mutex`, `nempty` e `nstored`. La variabile `nitems` rappresenta il numero di elementi da produrre e consumare ed è **letta da entrambi i thread**.

L'uso di una struttura permette di **organizzare e chiarire** che questi dati sono condivisi tra produttore e consumatore, e che i **semafori sono strumenti di sincronizzazione** usati per proteggere l'accesso concorrente al buffer.

Main

```
int
main(int argc, char **argv)
{
```

```

pthread_t    tid_produce, tid_consume;
if (argc != 2)
    {printf("usage: prodcons1 <#items>");exit(-1);}
nitems = atoi(argv[1]);
    /* crea i tre semafori */
shared.mutex = sem_open(SEM_MUTEX, O_CREAT | O_EXCL, FILE_MODE, 1);
shared.nempty = sem_open(SEM_NEMPTY, O_CREAT | O_EXCL, FILE_MODE, NBUFF);
shared.nstored = sem_open(SEM_NSTORED, O_CREAT | O_EXCL, FILE_MODE, 0);
    /* crea un thread produttore ed un thread consumatore */
pthread_create(&tid_produce, NULL, produce, NULL);
pthread_create(&tid_consume, NULL, consume, NULL);
    /* attende i due thread */
pthread_join(tid_produce, NULL);
pthread_join(tid_consume, NULL);
    /* rimuove i semafori */
sem_unlink(SEM_MUTEX);sem_unlink(SEM_NEMPTY);sem_unlink(SEM_NSTORED); exit(0);
}

```

Nel main, il programma **crea i tre semafori** POSIX con `sem_open()`, specificando `O_CREAT | O_EXCL` per garantire che siano creati nuovi e non già esistenti. Il semaforo `mutex` è inizializzato a 1 (per il mutuo accesso), `nempty` a `NBUFF` (posti vuoti iniziali) e `nstored` a 0 (nessun elemento ancora prodotto).

Dopo l'inizializzazione, il programma **crea due thread**: uno esegue la funzione `produce`, l'altro `consume`. Entrambi operano sul buffer condiviso e usano i semafori per sincronizzarsi. Infine, il main attende la **terminazione dei due thread** con `pthread_join` e **rimuove i semafori** con `sem_unlink()` per liberare le risorse.

Produttore

```
void *produce(void *arg)
{
    int i;

    for (i = 0; i < nitems; i++) {
        sem_wait(shared.empty); /* attende almeno un posto vuoto */

        sem_wait(shared.mutex);
        shared.buff[i % NBUFF] = i; /* memorizza i nel buffer circolare */
        sem_post(shared.mutex);

        sem_post(shared.nstored); /* un altro elemento è disponibile */
    }
    return (NULL);
}
```

La funzione del **produttore** ha il compito di inserire elementi nel **buffer condiviso**, uno alla volta, fino a raggiungere il numero totale di elementi specificato da `nitems`. A ogni iterazione, il produttore esegue un **`sem_wait` su `empty`** per accertarsi che ci sia almeno **uno slot vuoto** disponibile nel buffer. Questo evita di sovrascrivere dati non ancora consumati.

Successivamente, il produttore entra in **sezione critica** tramite **`sem_wait` sul semaforo `mutex`**, garantendo accesso esclusivo al buffer durante la scrittura. Il valore `i` viene quindi scritto nella posizione `i % NBUFF`, sfruttando l'indice circolare. Subito dopo, il **`mutex` viene rilasciato** con `sem_post`.

Infine, il produttore esegue un **`sem_post` su `nstored`**, incrementando il numero di elementi disponibili per il consumatore. Questo segnala che un **nuovo dato è pronto per essere letto**.

L'intera sequenza garantisce che l'accesso al buffer sia **sicuro e sincronizzato**, rispettando i vincoli del problema produttore-consumatore.

Consumatore

```
void *consume(void *arg)
{
    int i;

    for (i = 0; i < nitems; i++) {

        sem_wait(shared.nstored); /* attende almeno un elemento */

        sem_wait(shared.mutex);
        if (shared.buff[i % NBUFF] != i)
            printf("buff[%d] = %d\n", i, shared.buff[i % NBUFF]);
        sem_post(shared.mutex);
        sem_post(shared.nempty); /* un altro posto libero */
    }
    return (NULL);
}
```

La funzione del **consumatore** legge dal buffer condiviso tutti gli elementi prodotti, iterando fino a `nitems`. A ogni ciclo, esegue un **`sem_wait` su `nstored`** per assicurarsi che ci sia **almeno un elemento disponibile** da consumare. Se `nstored` è 0, il consumatore si blocca in attesa.

Quando è sicuro di poter procedere, entra in **sezione critica** con **`sem_wait` su `mutex`** per accedere al buffer in modo esclusivo. Legge il valore presente in `shared.buff[i % NBUFF]` e lo confronta con il valore atteso `i`, segnalando un eventuale errore tramite stampa su standard output.

Dopo la lettura, **rilascia il `mutex`** con `sem_post` e infine esegue un **`sem_post` su `nempty`**, comunicando al produttore che **è disponibile un nuovo posto vuoto nel buffer**.

Questa sequenza assicura che il consumatore non acceda al buffer in modo concorrente e che non legga mai da posizioni non ancora scritte, mantenendo la **corretta sincronizzazione** tra produttore e consumatore.

Deadlock

Nel contesto del problema produttore-consumatore, si può generare un **deadlock** se si **inverte l'ordine delle chiamate a `sem_wait()` nella funzione del consumatore**. Supponendo che il produttore inizi per primo, egli riempie il buffer con `NBUFF` elementi, portando il semaforo `nempty` a 0 (nessuno slot vuoto) e `nstored` a `NBUFF` (tutti i dati prodotti).

A questo punto, il produttore si blocca su `sem_wait(shared.nempty)` in attesa che il consumatore liberi spazio nel buffer.

Il consumatore inizia correttamente a consumare i primi `NBUFF` elementi, riducendo `nstored` a 0 e rialzando `nempty` a `NBUFF`. Tuttavia, se il consumatore **prima attende `mutex` e poi `nstored`**, può accadere che esso si blocchi **dopo aver acquisito il `mutex`**, ma **prima di poter ottenere `nstored`**, poiché il valore è 0.

Il risultato è che **entrambi i thread sono bloccati**: il produttore è in attesa di `mutex` (che è bloccato dal consumatore), mentre il consumatore è in attesa di `nstored`, che il produttore non può incrementare perché è fermo. Nessuno dei due può proseguire.

Questo è un classico esempio di **deadlock**, causato da un'errata **sequenza di acquisizione dei semafori**, che porta a **una dipendenza circolare non risolvibile**.

Semafori Posix basati su memoria

sem_init e sem_destroy

```
#include<semaphore.h>

int sem_init(sem_t *sem, int shared, unsigned int value);

int sem_destroy(sem_t *sem);
```

Oltre ai semafori POSIX **con nome**, che sono collegati a file nel filesystem, POSIX fornisce anche **semafori basati su memoria**. In questo caso, è l'applicazione a **fornire la memoria** in cui il semaforo sarà inizializzato e gestito.

L'inizializzazione avviene con la funzione `sem_init()`, che riceve tre argomenti: un **puntatore a sem_t**, un **flag shared** e un **valore iniziale**. Il significato del flag `shared` è cruciale:

- Se `shared` è **0**, il semaforo è **visibile solo ai thread dello stesso processo**.
- Se `shared` è **diverso da 0**, il semaforo è **condiviso tra più processi**, a condizione che sia collocato in una **memoria condivisa accessibile da tutti i processi coinvolti**.

Il terzo parametro `value` indica il **valore iniziale del semaforo** (ad esempio 1 per un semaforo binario).

Quando il semaforo non è più necessario, si libera la memoria associata usando la funzione `sem_destroy()`.

Esempio: produttore – consumatore

Come esempio pratico dell'uso dei **semafori basati su memoria**, si considera una variante del classico problema **produttore-consumatore**, in cui vi sono **più thread produttori** e un singolo consumatore.

Variabili Globali

```
#include      ...

#define NBUFF      10
#define MAXNTHREADS  100

int      nitems, nproducers;
/* sola lettura per produttori e consumatore */

struct { /* dati condivisi tra prods. e cons. */
    int  buff[NBUFF];
    int  nput;
    int  nputval;
    sem_t mutex, nempty, nstored; // semafori non puntatori
} shared;

void      *produce(void *), *consume(void *);
```

Le **variabili globali** `nitems` e `nproducers` definiscono rispettivamente il **numero totale di elementi** da produrre e il **numero di thread produttori**. Entrambe vengono impostate da **riga di comando**.

Nella struttura `shared` sono raccolti i **dati condivisi** tra produttori e consumatore. In particolare, vengono utilizzate due **nuove variabili**:

- `nput`, che rappresenta l'indice della **prossima posizione libera nel buffer**, calcolata in modo circolare con modulo `NBUFF`
- `nputval`, che contiene il **valore da scrivere nel buffer**

Queste due variabili servono per **coordinare i produttori** tra loro e assicurare che ciascuno inserisca dati corretti senza interferenze.

A differenza della versione con semafori con nome, qui i **semafori** (`mutex`, `nempty`, `nstored`) sono **contenuti direttamente nella struttura come variabili (non puntatori)**, perché si tratta di **semafori in memoria** inizializzati tramite `sem_init()`.

Main

```
int
main(int argc, char **argv)
{
    int            i, count[MAXNTHREADS];
    pthread_t      tid_produce[MAXNTHREADS], tid_consume;

    if (argc != 3)
        {printf("usage: prodcons3 <#items> <#producers>");exit(-1);}
    nitems = atoi(argv[1]);
    nproducers = MIN(atoi(argv[2]), MAXNTHREADS);

    /* inizializza tre semafori */
    sem_init(&shared.mutex, 0, 1);
    sem_init(&shared.nempty, 0, NBUFF);
    sem_init(&shared.nstored, 0, 0);

    /* crea tutti i produttori ed un consumatore */
    for (i = 0; i < nproducers; i++) {
        count[i] = 0;
        pthread_create(&tid_produce[i], NULL, produce, &count[i]);
    }
    pthread_create(&tid_consume, NULL, consume, NULL);

    /* aspetta tutti i produttori ed il consumatore */
    for (i = 0; i < nproducers; i++) {
        pthread_join(tid_produce[i], NULL);
        printf("count[%d] = %d\n", i, count[i]);
    }
    pthread_join(tid_consume, NULL);

    sem_destroy(&shared.mutex);
    sem_destroy(&shared.nempty);
    sem_destroy(&shared.nstored);
    exit(0);
}
```

Gli **argomenti della riga di comando** specificano il **numero totale di elementi** da produrre (`nitems`) e il **numero di thread produttori** da avviare (`nproducers`). Quest'ultimo è limitato a un massimo prefissato.

Vengono **inizializzati tre semafori** tramite la funzione `sem_init()`: un **mutex binario** inizializzato a 1, un **semaforo contatore** `nempty` inizializzato a `NBUFF` (spazi vuoti) e un **semaforo contatore** `nstored` inizializzato a 0 (nessun elemento ancora prodotto).

Per ciascun produttore viene creato un **thread**, a cui viene passato un contatore individuale. Viene poi avviato anche un **thread consumatore**.

Alla fine, il programma **attende la terminazione** di tutti i produttori e del consumatore tramite `pthread_join()`. I valori dei contatori vengono stampati per ogni produttore.

Infine, i **semafori vengono distrutti** con `sem_destroy()`, liberando le risorse di sistema.

Produttori

```
void *produce(void *arg)
{
    for ( ; ; ) {

        sem_wait(&shared.empty); /* aspetta almeno una locazione libera */

        sem_wait(&shared.mutex);

        if (shared.nput >= nitems) {
            sem_post(&shared.empty);
            sem_post(&shared.mutex);
            return(NULL);          /* tutto prodotto */
        }

        shared.buff[shared.nput % NBUFF] = shared.nputval;
        shared.nput++;
        shared.nputval++;

        sem_post(&shared.mutex);
        sem_post(&shared.nstored);
        /* un altro elemento memorizzato */
        *((int *) arg) += 1;
    }
}
```

La funzione `produce` è utilizzata da ciascun thread produttore per inserire dati in un buffer condiviso in modo sicuro e sincronizzato. All'inizio della funzione, si entra in un ciclo infinito che prosegue finché non vengono prodotti tutti gli elementi richiesti. Ogni thread produttore, prima di accedere al buffer, attende che ci sia almeno una **posizione libera** disponibile nel buffer stesso, utilizzando il semaforo `empty`, che tiene traccia degli slot vuoti. Successivamente, il thread acquisisce il **mutua esclusione** tramite il semaforo `mutex`, per garantire che l'accesso alle variabili condivise — in particolare `nput` (indice di inserimento nel buffer) e `nputval` (valore da inserire) — sia fatto da un solo thread alla volta.

Una volta ottenuto l'accesso esclusivo, il thread controlla se il numero totale di elementi richiesti (`nitems`) è già stato raggiunto. Se sì, rilascia i semafori e termina la sua esecuzione. Altrimenti, scrive nel buffer alla posizione `nput % NBUFF` (gestendo così il buffer in modalità **circolare**) il valore contenuto in `nputval`, poi incrementa entrambe le variabili (`nput` e `nputval`) per prepararsi all'inserimento successivo.

Terminata la scrittura, il thread **rilascia il mutex**, segnalando che ha finito l'uso della sezione critica, e poi incrementa il semaforo `nstored`, per indicare che è stato inserito un nuovo elemento disponibile per il consumo. Infine, aggiorna un contatore personale (passato come puntatore `arg`) che registra quanti elementi sono stati prodotti da quel singolo thread. Il ciclo poi riprende da capo, continuando fino alla produzione completa di tutti gli elementi richiesti.

Consumatore

```
void *consume(void *arg)
```

```

{
    int        i;
    for (i = 0; i < nitems; i++) {
        sem_wait(&shared.nstored); /* attende almeno un elemento memorizzato */

        sem_wait(&shared.mutex);

        if (shared.buff[i % NBUFF] != i)
            printf("error:buff[%d]=%d\n",i,shared.buff[i% NBUFF]);
        sem_post(&shared.mutex);
        sem_post(&shared.nempty); /* un altro posto libero */
    }
    return(NULL);
}

```

Nel codice del consumatore, la funzione `consume` ha il compito di **leggere gli elementi dal buffer condiviso** e verificare che i valori letti siano corretti. Il ciclo `for` si ripete per un numero di volte pari a `nitems`, cioè il numero totale di elementi che devono essere consumati.

Per ogni iterazione, il consumatore **attende che ci sia almeno un elemento disponibile nel buffer** eseguendo `sem_wait(&shared.nstored)`, che decrementa il semaforo `nstored` se il suo valore è maggiore di 0, oppure blocca l'esecuzione fino a che non lo diventa. Una volta sicuro che c'è qualcosa da leggere, il consumatore **acquisisce il semaforo mutex** tramite `sem_wait(&shared.mutex)` per **accedere in mutua esclusione** al buffer condiviso, evitando condizioni di race con altri thread (anche se in questo caso c'è solo un consumatore).

Successivamente, legge l'elemento nella posizione `i % NBUFF` del buffer. Il modulo serve a gestire il **buffer come struttura circolare**. Se il valore letto è diverso da quello atteso (`i`), stampa un messaggio di errore per indicare un'anomalia. Dopo la verifica, **rilascia il semaforo mutex** con `sem_post(&shared.mutex)` e **incrementa il semaforo nempty** con `sem_post(&shared.nempty)` per segnalare ai produttori che una posizione nel buffer è di nuovo libera.

Condivisione di semafori tra processi

Per condividere **semafori basati su memoria** tra processi, è essenziale che il semaforo, ovvero la variabile di tipo `sem_t`, sia allocato in una **zona di memoria condivisa** visibile a tutti i processi interessati. Inoltre, durante l'inizializzazione con `sem_init()`, il secondo argomento deve essere impostato a **1**, per indicare che il semaforo sarà usato tra processi diversi.

I semafori basati su memoria hanno in genere **persistenza di processo**, cioè esistono finché esiste il processo che li ha creati. Tuttavia, se il semaforo è collocato in memoria condivisa, la sua reale persistenza dipende da quanto a lungo quella porzione di memoria resta disponibile. Se invece il semaforo è creato con **shared = 0**, esso è visibile solo ai thread del processo che lo ha creato e scompare quando tale processo termina.

```
sem_t mysem;
sem_init(&mysem, 1, 0);
if (fork() == 0) {
    ...
    sem_post(&mysem);
}
sem_wait(&mysem);
```

Un errore comune consiste nel creare un semaforo all'interno della memoria privata di un processo (es. come variabile locale o globale non condivisa) e poi tentare di condividerlo con un processo figlio tramite una **fork()**. In tal caso, anche se il figlio eredita una copia del semaforo, questa **non è condivisa**, ma è una copia separata: le modifiche fatte dal processo figlio non influenzano il semaforo nel processo padre, invalidando così la sincronizzazione.

In alternativa, i **semafori con nome** possono essere condivisi tra processi diversi semplicemente facendo in modo che ogni processo li apra con lo stesso **nome** tramite la funzione **sem_open()**. Questo approccio è più semplice per la condivisione e non richiede la gestione esplicita di memoria condivisa.

Altro esempio

Questo esempio mostra come utilizzare un **semaforo Posix con nome** per sincronizzare due processi separati in modo semplice. I due processi sono creati tramite una chiamata a `fork()`: il **padre** (P1) esegue un'istruzione (S1) e, al termine, invoca `sem_post()` su un semaforo condiviso inizializzato a 0. Questo serve a sbloccare il processo **figlio** (P2), che all'inizio esegue `sem_wait()` sullo stesso semaforo, attendendo che S1 sia terminata prima di procedere con l'istruzione successiva (S2).

In questo esempio vogliamo eseguire S2 solo dopo che S1 è terminata (indipendentemente dallo scheduling dei processi).

Quindi

Nel processo P1 si inseriscono le istruzioni

```
...
S1;
sem_post(sincronizzazione);
```

Nel processo P2 le istruzioni

```
...
sem_wait(sincronizzazione);
S2;
```



```

#include ...
int main(int arg, char **argv){
    sem_t *sincronizzazione;
    int pid;
    sincronizzazione = sem_open("/test",O_CREAT,0666,0);
    pid = fork();
    if (pid==0){
        sem_wait(sincronizzazione);
        printf("di Sistemi Operativi\n");  /* Istruzione S2*/
        exit(0);
    }
    else {
        printf("Laboratorio ");           /* Istruzione S1 */
        sem_post(sincronizzazione);
        fflush(NULL);
        wait(NULL);
        sem_unlink("/test");
        exit(0);
    }
}

```

Nel dettaglio, il semaforo viene creato con `sem_open("/test", O_CREAT, 0666, 0)` ed è accessibile a entrambi i processi perché ha un **nome condiviso**. Quando il padre termina l'istruzione S1 ("Laboratorio") stampa e poi invoca `sem_post()`, consentendo al figlio di continuare con `sem_wait()` e stampare "di Sistemi Operativi\n", garantendo che l'ordine delle istruzioni sia rispettato, indipendentemente dallo scheduling dei processi. Alla fine, il semaforo viene rimosso con `sem_unlink()` dal processo padre. Questo esempio dimostra l'uso pratico di un **semaforo per la sincronizzazione tra processi**, garantendo il corretto ordine di esecuzione.